



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Algoritmi e strutture dati

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

*Appunti redatti per gli studenti del DISI.
Eventuali errori possono essere segnalati su [GitHub](#).*

settembre 2026

Indice

1	Introduzione al Corso	6
1.1	Organizzazione e Obiettivi	6
1.2	Programma del Corso	6
2	Introduzione all'Algoritmica: Il Problema della Somma Massimale	7
2.1	Cenni Storici e Applicazioni	7
2.2	Evoluzione delle Soluzioni Algoritmiche	7
2.2.1	Versione 1: Forza Bruta $\mathcal{O}(n^3)$	7
2.2.2	Versione 2: Ottimizzazione con Accumulatore $\mathcal{O}(n^2)$	8
2.2.3	Versione 3: Divide-et-Impera $\mathcal{O}(n \log n)$	8
2.2.4	Versione 4: Programmazione Dinamica (Algoritmo di Kadane) $\mathcal{O}(n)$	9
3	Problemi Computazionali e Algoritmi	9
3.1	Cenni Storici e Origine del Nome	9
3.2	Pseudo-Codice	10
4	Esempi Noti: Minimo e Ricerca	11
4.1	Problema 1: Ricerca del Minimo	11
4.1.1	Algoritmo Naïf $\mathcal{O}(n^2)$	11
4.1.2	Algoritmo Ottimo $\mathcal{O}(n)$	11
4.2	Problema 2: Ricerca in un Array (Lookup)	11
4.2.1	Ricerca Lineare Naïf $\mathcal{O}(n)$	11
4.2.2	Ricerca Binaria o Dicotomica $\mathcal{O}(\log n)$	12
5	Valutazione degli Algoritmi: Efficienza e Correttezza	13
5.1	Analisi dell'Efficienza (Complessità Computazionale)	13
5.2	Dimostrazione di Correttezza	13
6	Analisi degli Algoritmi: Modelli di Calcolo e Complessità	14
6.1	Obiettivi dell'Analisi	14
6.2	Definizione di Complessità	14
6.3	Modelli di Calcolo Astratti	14
6.4	Relazioni di Ricorrenza	14
7	Classi di Complessità e Notazione Asintotica	15
7.1	Ordini di Crescita Comuni	15
7.2	Definizioni Formali	15
7.2.1	Notazioni per Limiti Non Stretti	16
7.3	Proprietà Matematiche della Notazione Asintotica	16
7.4	Regole Pratiche e semplificazioni	16
8	Complessità: Algoritmi vs. Problemi	17
8.1	Esempio: La Moltiplicazione tra Numeri	17
9	Algoritmi di Ordinamento (Sorting)	18
9.1	Selection Sort	18
9.2	Insertion Sort	18
9.3	Merge Sort	18

10 Limiti Teorici dell'Ordinamento e Modelli Alternativi	19
10.1 Limite Inferiore per l'Ordinamento Basato su Confronti	19
10.2 Counting Sort: Superare il limite cambiando Modello	19
11 Risoluzione delle Equazioni di Ricorrenza	19
11.1 1. Analisi per Livelli (Albero delle Chiamate)	20
11.2 2. Metodo della Sostituzione (o per Tentativi)	20
11.3 3. Il Teorema dell'Esperto (Master Theorem)	21
11.3.1 Versione Estesa e Casi Logaritmici	21
11.4 Altre Ricorrenze Comuni (Lineari)	22
12 Ritorno alla Somma Massimale: Analisi Formale	23
13 Analisi Ammortizzata	23
13.1 Caso di Studio: Contatore Binario	24
13.1.1 1. Metodo dell'Aggregazione	24
13.1.2 2. Metodo degli Accantonamenti	24
13.1.3 3. Metodo del Potenziale	25
14 Vettori Dinamici (Dynamic Arrays)	26
14.1 Politiche di Espansione	26
14.1.1 1. Incremento Fisso (es. + Costante d)	26
14.2 Costo ammortizzato: $T(n)/n = \mathcal{O}(n)$	26
14.2.1 2. Raddoppiamento (es. $\times 2$)	26
14.3 Cancellazione e Contrazione	26
15 Applicazioni Pratiche e Conclusioni	28
16 Strutture di Dati Astratte: Definizioni Fondamentali	28
16.1 Specifica vs Implementazione	28
16.2 Caratteristiche delle Strutture Dati	28
17 Tipi di Dato Astratto Principali	30
17.1 1. Sequenza	30
17.2 2. Insiemi (Set)	30
17.3 3. Dizionari (Mappe / Array Associativi)	30
17.4 4. Alberi e Grafi	31
18 Implementazione delle Strutture Dati Elementari	32
18.1 Lista (Linked List)	32
18.2 Pila (Stack)	32
18.3 Coda (Queue)	33
19 Alberi	33
19.1 Introduzione e Definizioni Formali	33
19.1.1 Terminologia e Metriche	34
19.2 Alberi Binari	35
19.2.1 Implementazione in Memoria	35
19.3 Visite di Alberi (Traversals)	35
19.3.1 1. Visita in Profondità (Depth-First Search - DFS)	35

19.3.2	2. Visita in Ampiezza (Breadth-First Search - BFS)	36
19.4	Alberi Generici	37
19.4.1	Visite per Alberi Generici	37
19.4.2	Implementazioni in Memoria per Alberi Generici	37
20	Alberi Binari di Ricerca (ABR)	38
20.1	Struttura del Nodo	38
20.2	Operazioni di Base	38
20.2.1	Ricerca (Lookup)	38
20.2.2	Ricerca del Minimo e del Massimo	39
20.2.3	Successore e Predecessore	39
20.3	Inserimento e Cancellazione	39
20.3.1	Inserimento	39
20.3.2	Cancellazione	39
20.4	Costo Computazionale degli ABR	40
21	Alberi Binari di Ricerca Bilanciati: Alberi Red-Black	41
21.1	Altezza Nera e Teoremi	41
21.2	Meccanismi di Ribilanciamento: Le Rotazioni	41
21.3	Inserimento negli Alberi Red-Black	42
21.4	Cancellazione negli Alberi Red-Black	42
22	Hashing e Dizionari	42
22.1	Motivazioni e Implementazione Ideale	42
22.1.1	Tabelle ad Accesso Diretto e Collisioni	43
23	Progettazione delle Funzioni Hash	44
23.1	Metodi Comuni per Funzioni Hash Semplici	44
24	Gestione delle Collisioni	45
24.1	1. Liste/Vettori di Trabocco (Chaining)	45
24.2	2. Indirizzamento Aperto (Open Addressing)	45
24.2.1	Tecniche di Ispezione	45
24.2.2	La Cancellazione nell'Indirizzamento Aperto	46
25	Reality Check e Considerazioni Pratiche	47
25.1	Implementazioni nel Mondo Reale	47
25.2	Limiti dell'Hashing	47
26	Insiemi e Dizionari: Riassunto delle Implementazioni	47
26.1	Insiemi realizzati con Vettori Booleani	48
26.2	Insiemi realizzati con Liste o Vettori Non Ordinati	49
26.3	Insiemi realizzati con Liste o Vettori Ordinati	49
26.4	Insiemi tramite Strutture Dati Complesse	50
26.5	Riepilogo Asintotico delle Implementazioni	50
26.6	Costi Computazionali in Python	50
27	Grafi	51
27.1	Definizioni Formali	51

28 Memorizzazione e Implementazione	52
28.1 Matrice di Adiacenza	52
28.2 Liste di Adiacenza (o Vettori di Adiacenza)	52
29 Visite dei Grafi (Graph Traversal)	53
29.1 Visita in Ampiezza (Breadth-First Search - BFS)	53
29.2 Visita in Profondità (Depth-First Search - DFS)	53
30 Applicazioni Pratiche della DFS	54
30.1 1. Componenti Connesse (Grafì Non Orientati)	54
30.2 2. Ciclicità e Grafi Aciclici (DAG)	54
30.3 3. Ordinamento Topologico (solo per DAG)	54
30.4 4. Componenti Fortemente Connesse (SCC - Grafi Orientati)	54
31 Strutture Dati Speciali: Code con Priorità	55
31.1 Specifica del Tipo di Dato	55
31.2 L'Albero Heap	55
31.2.1 Memorizzazione su Vettore (Vettore Heap)	56
31.3 L'algoritmo HeapSort	56
31.3.1 1. Ripristino della Proprietà Heap (<code>maxHeapRestore</code>)	56
31.3.2 2. Costruzione dell'Heap (<code>heapBuild</code>)	56
31.3.3 3. Algoritmo di Ordinamento Finale (<code>heapsort</code>)	57
31.4 Implementazione della Coda con Priorità (Min-Heap)	57
32 Insiemi Disgiunti (Merge-Find Set)	58
32.1 1. Realizzazione basata su Liste	58
32.2 2. Realizzazione basata su Alberi (Foresta)	58
32.3 Ottimizzazioni: Euristiche	58
32.3.1 Euristiche sul Peso (per Liste)	58
32.3.2 Euristiche sul Rango (per Alberi)	59
32.3.3 Euristiche della Compressione dei Cammini (per Alberi)	59
32.3.4 Combinazione: Rango + Compressione	59
33 Il Problema dei Cammini Minimi e la Scelta della Struttura Dati	59
33.1 Caratteristiche delle Soluzioni	60
34 Schema Generico e Teorema di Bellman	61
34.1 Il Teorema di Bellman	61
34.2 Algoritmo Prototipo e Rilassamento	61
35 Algoritmi per Sorgente Singola (SSSP)	62
35.1 Algoritmo di Dijkstra (1959) - Pesi Positivi	62
35.2 Algoritmo di Bellman-Ford-Moore (1958) - Pesi Negativi	62
35.3 Caso Speciale: Grafi Orientati Aciclici (DAG)	62
36 Cammini Minimi per Tutte le Coppie (APSP)	63
36.1 Algoritmo di Floyd-Warshall (1962)	63
36.2 Chiusura Transitiva (Algoritmo di Warshall)	63
37 Metodologia di Risoluzione dei Problemi	64

37.1	Classificazione dei Problemi	64
38	La Tecnica del Divide-et-Impera	65
38.1	Esempio Introduttivo: Torri di Hanoi	65
39	Quicksort (Hoare, 1961)	66
39.1	Implementazione	66
39.2	Analisi della Complessità	66
40	Algoritmo di Strassen (Moltiplicazione di Matrici)	68
41	Applicazione Pratica: Ricerca di un Gap	69
42	Programmazione Dinamica	70
42.1	Approccio Metodologico	70
43	Esempi Classici	71
43.1	1. Il Problema del Domino	71
43.2	2. Il Problema di Hateville	71
43.3	3. Il Problema dello Zaino (Knapsack 0/1)	72
44	Programmazione Dinamica: Memoization e Varianti	73
44.1	Zaino 0/1 con Memoization	73
44.2	Variante dello Zaino, Senza Limiti (Unbounded Knapsack)	73
44.2.1	Semplificazione ad un vettore unidimensionale	73
45	La Sottosequenza Comune Massimale (LCS)	75
45.1	Sottostruttura Ottima e Formula Ricorsiva	75
45.1.1	Algoritmo Dinamico	75
45.2	Ricostruzione della Soluzione e Ottimizzazione	76
45.3	Reality Check: Il tool <code>diff</code>	76
46	Programmazione Dinamica: Applicazioni Avanzate	77
46.1	Prodotto di Catena di Matrici	77
46.1.1	Parentesizzazione e Numeri di Catalan	77
46.1.2	Sottostruttura Ottima e Formula Ricorsiva	77
46.1.3	Algoritmo Bottom-Up	77
46.2	Insieme Indipendente di Intervalli Pesati	79
46.2.1	Pre-elaborazione Necessaria	79
46.2.2	Formula DP e Algoritmo	79
46.3	String Matching Approssimato	80
46.3.1	Sottostruttura Ottima	80
46.3.2	Algoritmo e Take-home Message	80

1 Introduzione al Corso

1.1 Organizzazione e Obiettivi

- **Docente del corso:** Prof. Alberto Montresor.
- **Obiettivi principali:** Imparare ad analizzare il codice degli algoritmi, convincersi del loro funzionamento e implementarli, oltre ad assimilare tecniche standard per risolvere problemi complessi.
- **Valutazione:** L'esame finale è suddiviso in una parte scritta (50
- **Libro di testo adottato:** *Algoritmi e Strutture di Dati* di Bertossi e Montresor (Terza edizione).

1.2 Programma del Corso

Il corso è diviso in due moduli:

- **Modulo A:** Analisi degli algoritmi (notazione asintotica, ricorrenze, analisi ammortizzata), strutture dati elementari (alberi, grafi, insiemi e dizionari), code con priorità e tecnica del *divide-et-impera*.
- **Modulo B:** Strutture dati avanzate (insiemi disgiunti), programmazione dinamica, algoritmi greedy, ricerca locale, backtrack, algoritmi probabilistici e studio dei problemi NP-completi.

2 Introduzione all'Algoritmica: Il Problema della Somma Massimale

Per introdurre l'importanza dell'efficienza degli algoritmi, viene presentato un problema classico: la ricerca del sottovettore di somma massimale.

Definizione del Problema: Somma Massimale

Input: Un vettore A contenente n interi (sia negativi che positivi). **Output:** La somma massimale, definita come il valore massimo che si può ottenere sommando gli elementi di un qualunque sottovettore contiguo di A (ovvero una sequenza di elementi consecutivi).

2.1 Cenni Storici e Applicazioni

- Il problema fu introdotto nel 1977 da **Ulf Grenander** come versione semplificata per il *maximum likelihood in image processing* (immagini 2D).
- Nel 1984, **Jay Kadane** propose un algoritmo lineare per la sua risoluzione.
- **Applicazioni pratiche:** Viene utilizzato nell'analisi delle sequenze genomiche per individuare segmenti biologicamente significativi (es. regioni conservate), assegnando un punteggio (score) a ciascun residuo e cercando il segmento con somma massima.

2.2 Evoluzione delle Soluzioni Algoritmiche

Di seguito vengono presentate quattro diverse soluzioni per risolvere il problema, mostrando come ottimizzazioni successive riducano drasticamente il tempo di esecuzione.

2.2.1 Versione 1: Forza Bruta $O(n^3)$

L'approccio più banale consiste nel ciclare su tutte le possibili coppie di indici (i, j) con $i \leq j$.

- Si calcola la somma per ogni possibile sottovettore chiamando una funzione `sum()` o usando un terzo ciclo esplicito.
- Si aggiorna il valore massimo trovato finora (`maxSoFar`).

```
// Codice Java - Soluzione  $O(n^3)$  con ciclo esplicito
int maxsum1(int[] A, int n) {
    int maxSoFar = 0; // Massimo trovato finora
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int sumSoFar = 0; // Accumulatore per il sottovettore
            for (int k = i; k <= j; k++) {
                sumSoFar = sumSoFar + A[k];
            }
            maxSoFar = Math.max(maxSoFar, sumSoFar);
        }
    }
}
```

```

}
return maxSoFar;
}

```

2.2.2 Versione 2: Ottimizzazione con Accumulatore $\mathcal{O}(n^2)$

Invece di ricalcolare la somma da zero ogni volta, si sfrutta un'ottimizzazione: se è nota la somma s dei valori in $A[i...j]$, la somma di $A[i...j + 1]$ è semplicemente $s + A[j + 1]$.

Codice Python - Soluzione $\mathcal{O}(n^2)$

```

def maxsum2(A):
    maxSoFar = 0 # Massimo trovato finora
    for i in range(0, len(A)):
        sum = 0 # Accumulatore
        for j in range(i, len(A)):
            sum = sum + A[j]
        maxSoFar = max(maxSoFar, sum)
    return maxSoFar

```

*Nota del trascrittore: Sulle slide è mostrata anche un'implementazione Python $\mathcal{O}(n^2)$ che fa uso della funzione di libreria **accumulate** del modulo **itertools**, la quale risulta più veloce in pratica grazie all'implementazione sottostante in C.*

2.2.3 Versione 3: Divide-et-Impera $\mathcal{O}(n \log n)$

L'algoritmo suddivide il problema in sottoproblemi più piccoli:

- Si divide il vettore a metà.
- Si calcola la somma massimale nella parte sinistra (**maxL**).
- Si calcola la somma massimale nella parte destra (**maxR**).
- Si calcola la somma massimale del sottovettore che "attraversa" la metà, unendo il massimo a sinistra che termina in m (**maxLL**) e il massimo a destra che inizia in $m + 1$ (**maxRR**).
- Si ritorna il massimo tra questi tre valori.

Frammento logico in Python - Divide et Impera

```

# Ritorna il massimo tra parte sx, parte dx e la somma incrociata al centro

return max(maxL, maxR, maxLL + maxRR)

```

2.2.4 Versione 4: Programmazione Dinamica (Algoritmo di Kadane) $\mathcal{O}(n)$

La soluzione ottima lineare si basa sulla programmazione dinamica. Definiamo $maxHere_i$ come il valore del sottovettore di somma massima che termina *esattamente* in posizione $A[i]$.

La formula ricorsiva è:

$$maxHere_i = \begin{cases} max(A[0], 0) & i = 0 \\ max(maxHere_{i-1} + A[i], 0) & i > 0 \end{cases}$$

Il risultato finale è il valore massimo contenuto tra tutti i valori calcolati $maxHere_0 \dots maxHere_{n-1}$.

```
// Codice Java - Soluzione ottima  $\mathcal{O}(n)$ 
int maxsum4(int[] A, int n) {
    int maxSoFar = 0;
    int maxHere = 0;
    for (int i = 0; i < n; i++) {
        maxHere = Math.max(maxHere + A[i], 0);
        maxSoFar = Math.max(maxSoFar, maxHere);
    }
    return maxSoFar;
}
```

*Nota aggiuntiva dal materiale: Esiste una variante Python di questo algoritmo in $\mathcal{O}(n)$ che, oltre a calcolare il valore della somma massima, tiene traccia e restituisce gli indici **start** ed **end** (inizio e fine) del sottovettore trovato.*

3 Problemi Computazionali e Algoritmi

Definizioni Fondamentali

Problema Computazionale: Dati un dominio di input e un dominio di output, è la relazione matematica che associa ogni elemento dell'input ad uno o più elementi dell'output.

Algoritmo: Dato un problema computazionale, è un procedimento *effettivo*, espresso tramite un insieme di passi elementari ben specificati in un sistema formale di calcolo, che risolve il problema in un tempo finito.

3.1 Cenni Storici e Origine del Nome

- I primi algoritmi documentati risalgono all'antichità: il Papiro di Rhind (1850 a.C. circa) conteneva l'algoritmo del contadino per la moltiplicazione. I matematici greci e babilonesi formularono procedure note ancor oggi, come l'algoritmo di Euclide per il massimo comune divisore (MCD).
- **Algoritmo:** La parola deriva dal nome del matematico, astronomo e astrologo **al-Khwarizmi** (nato in Uzbekistan, operò a Baghdad). L'opera "Algoritmi de numero indorum" introdusse i numeri arabi nel mondo occidentale (dal termine arabo *sifr* = 0, poi tradotto come *zephirum*, deriva *zero* e *cifra*).

- **Algebra:** Deriva dal titolo del suo libro del 820 d.C., "Al-Kitab al-muhtasar fi hisab al-gabr wa-l-muqabala".

3.2 Pseudo-Codice

Il linguaggio naturale è spesso troppo impreciso per descrivere in modo non ambiguo una procedura logica. Per questo motivo viene introdotto lo **pseudo-codice**, un linguaggio intermedio e formale che astrae dalla specifica sintassi di linguaggi di programmazione come C, Java o Python.

Regole principali e convenzioni per lo pseudo-codice adottato nel corso:

- Assegnamenti: `a = b`, `a += b`, scambio `a, b = b, a` o con variabile temporanea.
- Dichiarazioni di array: `T[] A = new T[0...n-1] = {0}` per un array 1D, `T[][] B = new T[0...]` per una matrice.
- Operatori logici testuali: `and`, `or`, `not`.
- Operatori matematici: `+`, `-`, `*`, `/`, `[x]`, `⌈x⌉`, `log`, `x2`.
- Istruzioni condizionali: `if condizione then ... else ...`
- Cicli iterativi: `while ... do`, `foreach ... in ... do`, `for indice = inizio to fine do`.
- Commenti: Segnalati dal simbolo `%`.
- Gestione Oggetti: `r = new RECTANGLE`, `delete r`, `r = nil`.

4 Esempi Noti: Minimo e Ricerca

Analizziamo due problemi classici confrontando l'approccio *naïf* e ingenuo con la soluzione *efficiente*.

4.1 Problema 1: Ricerca del Minimo

Problema: Trovare il valore minimo all'interno di un insieme S . Definizione matematica: $\min(S) = a \Leftrightarrow \exists a \in S : \forall b \in S : a \leq b$

4.1.1 Algoritmo Naïf $\mathcal{O}(n^2)$

Confronta ogni elemento con *tutti* gli altri per verificare se è strettamente il minore. Il costo in termine di operazioni rilevanti (confronti) è $n^2 - n$.

```
% Algoritmo Naïf per il minimo (n^2 - n confronti)
int min(int[] S, int n)
for i = 0 to n - 1 do
  boolean isMin = true
  for j = 0 to n - 1 do
    if i != j and S[j] < S[i] then
      isMin = false
  if isMin then
    return S[i]
```

4.1.2 Algoritmo Ottimo $\mathcal{O}(n)$

Si tiene traccia di un "minimo parziale" iterando e aggiornandolo mano a mano. Il costo è di $n - 1$ confronti.

```
% Algoritmo Efficiente per il minimo (n - 1 confronti)
int min(int[] A, int n)
int minSoFar = A[0]
for i = 1 to n - 1 do
  if A[i] < minSoFar then
    minSoFar = A[i]
return minSoFar
```

4.2 Problema 2: Ricerca in un Array (Lookup)

Problema: Dato un vettore A **ordinato** e privo di duplicati ($a_0 < a_1 < \dots < a_{n-1}$), e un valore v , restituire l'indice i in cui si trova v , oppure -1 se non esiste.

4.2.1 Ricerca Lineare Naïf $\mathcal{O}(n)$

Si scorre tutto l'array. L'operazione richiede nel caso peggiore n confronti.

```
int lookup(int[] A, int n, int v)
for i = 0 to n - 1 do
  if A[i] == v then
    return i
return -1
```

4.2.2 Ricerca Binaria o Dicotomica $\mathcal{O}(\log n)$

Poiché l'array è ordinato, si analizza l'elemento centrale (all'indice m). Se non è il numero cercato, si restringe il campo di ricerca dimezzando l'array iterativamente (a sinistra se $v < A[m]$, a destra se $v > A[m]$). In questo modo eseguiamo, nel caso pessimo, $2\lceil \log_2 n \rceil$ operazioni.

% Versione Ricorsiva

```
int bsRec(int[] A, int v, int start, int end)
```

```
if start > end then
```

```
return -1 % Vettore vuoto o elemento non trovato
```

```
int m = floor(start + (end - start) / 2) % Calcolo indice centrale sicuro da overflow
```

```
if A[m] == v then
```

```
    return m % Trovato!
```

```
else if A[m] < v then
```

```
    return bsRec(A, v, m + 1, end) % Sottovettore di destra
```

```
else
```

```
    return bsRec(A, v, start, m - 1) % Sottovettore di sinistra
```

*Nota: Sulle slide è stato fatto notare che calcolare l'indice medio come $m = (start+end)/2$ può portare a un bug noto come **integer overflow** per input sufficientemente grandi. L'operazione corretta e sicura in logica pseudo-codice è $m = start + (end - start)/2$.*

5 Valutazione degli Algoritmi: Efficienza e Correttezza

5.1 Analisi dell'Efficienza (Complessità Computazionale)

La complessità di un algoritmo analizza le risorse impiegate per la risoluzione, espressa in funzione della dimensione dell'input. Le due metriche principali sono:

- **Spazio:** Memoria utilizzata durante il processo.
- **Tempo:** Non calcolato in "wall-clock time" (tempo d'orologio), poiché influenzato da hardware, sistema operativo, linguaggio e bravura del programmatore, ma stimato contando il **numero di operazioni rilevanti** o dominanti (es. i confronti per il minimo o la ricerca, o gli accessi alla memoria).

Questo approccio astrae dalle limitazioni del calcolatore per fornire una formula generalizzata della prestazione pura dell'algoritmo.

5.2 Dimostrazione di Correttezza

Oltre all'efficienza, dobbiamo garantire (tramite dimostrazione matematica formale) che l'algoritmo produca sempre il risultato corretto. Per fare questo per gli algoritmi *iterativi*, ci si avvale del concetto di **invariante di ciclo**.

Definizione: Invariante di Ciclo

Un invariante di ciclo è una proposizione (una condizione logica) che è **sempre vera** all'inizio dell'iterazione di un ciclo.

La dimostrazione si basa sul principio di induzione e segue tre passi:

1. **Inizializzazione (caso base):** La condizione è vera prima che inizi la prima esecuzione del ciclo.
2. **Conservazione (passo induttivo):** Se la condizione è vera prima di una generica esecuzione del ciclo, rimarrà vera anche alla fine (ovvero, prima del ciclo successivo).
3. **Conclusione:** Alla terminazione del ciclo, unito all'invariante dimostrato, si ottiene la garanzia logica che l'algoritmo ha calcolato il valore desiderato e corretto.

Esempio dell'invariante per l'algoritmo del minimo parziale: "*All'inizio di ogni iterazione del ciclo for (riga 3), la variabile `minSoFar` contiene sempre il valore del minimo parziale degli elementi $A[0...i-1]$* ".

Per gli algoritmi *ricorsivi* (come la Binary Search), si utilizza invece la dimostrazione per induzione standard (sulla grandezza n dell'input):

- **Caso base:** Ad es. $n = 0$, il sottovettore è vuoto e viene restituito -1.
- **Ipotesi induttiva:** Si ipotizza vera l'asserzione per i casi $n' < n$.
- **Passo induttivo:** Si dimostra che, se vale per i sottomultipli (chiamate ricorsive), l'output per n è matematicamente corretto.

6 Analisi degli Algoritmi: Modelli di Calcolo e Complessità

6.1 Obiettivi dell'Analisi

L'obiettivo primario dell'analisi di un algoritmo è stimare la sua efficienza, solitamente in termini di tempo. Questa stima serve a:

- Capire quanto tempo impiegherà il programma per un dato input.
- Valutare la dimensione massima dell'input gestibile in tempi ragionevoli.
- Confrontare e scegliere l'algoritmo migliore tra più alternative.
- Ottimizzare i colli di bottiglia.

6.2 Definizione di Complessità

La complessità è una funzione matematica non negativa che lega la **dimensione dell'input** alle risorse impiegate:

- **Spazio:** $S : \text{"Dimensione input"} \rightarrow \text{"Spazio in memoria"}$
- **Tempo:** $T : \text{"Dimensione input"} \rightarrow \text{"Tempo di calcolo"}$

La dimensione dell'input varia a seconda del problema (es. numero di elementi in un vettore, numero di bit di un intero).

6.3 Modelli di Calcolo Astratti

Per definire matematicamente il costo di un algoritmo (es. il numero di "istruzioni elementari"), occorre un modello computazionale teorico che bilanci astrazione e realismo.

- **Macchina di Turing:** Utile per la calcolabilità, ma di livello troppo basso per l'analisi della complessità degli algoritmi pratici.
- **Random Access Machine (RAM):** È il modello di riferimento per il corso.
 - Ha un processore singolo.
 - Memoria ad accesso diretto (costante) divisa in parole di w bit.
 - Il costo delle istruzioni aritmetico/logiche elementari su singole parole di macchina è assunto **costante**.

6.4 Relazioni di Ricorrenza

Nel calcolare il tempo per funzioni ricorsive (es. Binary Search), si formula una **relazione di ricorrenza**. Schema generale:

$$T(n) = \begin{cases} \text{costo del caso base} & \text{se } n \leq n_0 \\ \text{costo chiamate ricorsive} + \text{lavoro locale} & \text{se } n > n_0 \end{cases}$$

Ad esempio, l'equazione di ricorrenza per la ricerca dicotomica è $T(n) = T(n/2) + d$ per $n \geq 1$ (con $T(0) = c$). Espandendola iterativamente fino a $n = 2^k$, risulta una complessità $T(n) = d \log n + e$.

7 Classi di Complessità e Notazione Asintotica

L'analisi asintotica studia il comportamento della funzione di costo $T(n)$ al tendere all'infinito della dimensione dell'input n , ignorando costanti moltiplicative e termini di ordine inferiore.

7.1 Ordini di Crescita Comuni

Gli ordini di grandezza più diffusi, dal più veloce al più lento, sono:

1. Costante: $\mathcal{O}(1)$
2. Logaritmica: $\mathcal{O}(\log n)$
3. Lineare: $\mathcal{O}(n)$
4. Loglineare: $\mathcal{O}(n \log n)$
5. Quadratica: $\mathcal{O}(n^2)$
6. Cubica: $\mathcal{O}(n^3)$
7. Esponenziale: $\mathcal{O}(2^n)$ o simili
8. Fattoriale: $\mathcal{O}(n!)$

7.2 Definizioni Formali

Notazione $\mathcal{O}$ (Big-O) - Limite Superiore

Indichiamo con $\mathcal{O}(g(n))$ l'insieme delle funzioni $f(n)$ per cui:

$$\exists c > 0, \exists m \geq 0 : f(n) \leq c \cdot g(n), \forall n \geq m$$

Significato: $f(n)$ cresce al più come $g(n)$.

Notazione Ω (Omega) - Limite Inferiore

Indichiamo con $\Omega(g(n))$ l'insieme delle funzioni $f(n)$ per cui:

$$\exists c > 0, \exists m \geq 0 : f(n) \geq c \cdot g(n), \forall n \geq m$$

Significato: $f(n)$ cresce almeno quanto $g(n)$.

Notazione Θ (Theta) - Limite Stretto

Indichiamo con $\Theta(g(n))$ l'insieme delle funzioni $f(n)$ per cui:

$$\exists c_1 > 0, \exists c_2 > 0, \exists m \geq 0 : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \forall n \geq m$$

Significato: $f(n)$ cresce *esattamente* come $g(n)$ a meno di costanti. NB: $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n))$.

7.2.1 Notazioni per Limiti Non Stretti

- **Piccolo-O** (o): Limite superiore *stretto* (strettamente minore asintoticamente).
- **Piccolo-Omega** (ω): Limite inferiore *stretto* (strettamente maggiore asintoticamente).

Utile il calcolo tramite i limiti per $n \rightarrow \infty$:

- $\lim \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = o(g(n))$
- $\lim \frac{f(n)}{g(n)} = c > 0 \Rightarrow f(n) = \Theta(g(n))$
- $\lim \frac{f(n)}{g(n)} = +\infty \Rightarrow f(n) = \omega(g(n))$

7.3 Proprietà Matematiche della Notazione Asintotica

Siano date delle costanti e delle generiche funzioni di costo:

- **Dualità:** $f(n) = \mathcal{O}(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$.
- **Eliminazione Costanti:** $f(n) = \mathcal{O}(g(n)) \Leftrightarrow a \cdot f(n) = \mathcal{O}(g(n)), \forall a > 0$.
- **Somma (blocchi in sequenza):** L'ordine di grandezza di una sequenza è dominato dal termine più oneroso:

$$f_1(n) + f_2(n) = \mathcal{O}(\max(g_1(n), g_2(n))) \quad \text{se} \quad f_i(n) = \mathcal{O}(g_i(n))$$

- **Prodotto (cicli annidati):**

$$f_1(n) \cdot f_2(n) = \mathcal{O}(g_1(n) \cdot g_2(n))$$

- **Simmetria e Transitività:** Valgono le proprietà transitiva per tutte le notazioni e di simmetria per il Θ .

7.4 Regole Pratiche e semplificazioni

- **Polinomi:** Qualsiasi polinomio di grado k , indipendentemente dai coefficienti (purché $a_k > 0$), è $\Theta(n^k)$.
- **Logaritmi:** Qualsiasi base logaritmica è influente ai fini asintotici ($\log_a n = \Theta(\log n)$). Inoltre, il logaritmo cresce sempre più lentamente di un *qualsiasi* polinomio: $\log n = \mathcal{O}(n^k) \forall k > 0$.
- **Esponenziali:** Basi diverse negli esponenziali non sono equivalenti: $2^n \neq \Theta(3^n)$ e anzi $2^n = \mathcal{O}(3^n)$.

8 Complessità: Algoritmi vs. Problemi

Esiste una netta distinzione tra valutare l'efficienza di uno specifico *algoritmo* e valutare l'efficienza intrinseca del *problema* in sé.

Complessità di un Problema Computazionale

- **Limite superiore $\mathcal{O}(f(n))$:** Un problema ha limite superiore $\mathcal{O}(f(n))$ se esiste almeno un algoritmo corretto in grado di risolverlo in tempo $\mathcal{O}(f(n))$.
- **Limite inferiore $\Omega(g(n))$:** Un problema ha limite inferiore $\Omega(g(n))$ se ogni possibile algoritmo (anche quelli non ancora inventati) richiede asintoticamente tempo $\Omega(g(n))$.

Se il limite inferiore e il limite superiore coincidono, l'algoritmo che lo raggiunge è considerato **asintoticamente ottimo**.

8.1 Esempio: La Moltiplicazione tra Numeri

Spesso tendiamo a valutare un problema basandoci sull'algoritmo naif che ci è stato insegnato.

- L'algoritmo elementare per la somma di due numeri binari di n bit costa $\Theta(n)$.
- L'algoritmo elementare per il prodotto di due numeri binari genera n prodotti parziali per n bit e costa $\Theta(n^2)$.

Questo non significa che la moltiplicazione debba per forza costare $\Theta(n^2)$. È *l'algoritmo classico* a costare così, non il problema.

Nel 1960, Kolmogorov ipotizzò che il limite inferiore del prodotto fosse $\Omega(n^2)$. Una settimana dopo, il suo studente Karatsuba smentì questa teoria inventando un algoritmo Divide-et-impera (che suddivide il prodotto riducendo il numero di chiamate ricorsive da 4 a 3), abbassando il limite superiore a $\mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$. Nel corso dei decenni, nuovi algoritmi (Toom-Cook, FFT) hanno progressivamente abbassato questo limite. Il limite inferiore dimostrato ad oggi è solo $\Omega(n)$.

9 Algoritmi di Ordinamento (Sorting)

L'ordinamento è fondamentale per comprendere come il comportamento asintotico di un algoritmo dipenda pesantemente dalla conformazione dell'input. Per gli algoritmi di ordinamento, si analizza il caso *Peggior*e, *Medio* e *Migliore*.

9.1 Selection Sort

Cerca il minimo della parte non ordinata e lo sostituisce, iterando la parte restante.

- **Costo:** Il numero di confronti è fisso: $\sum_{i=0}^{n-2} (n - i - 1) = \frac{n(n-1)}{2} = \Theta(n^2)$.
- **Comportamento:** Non sfrutta ordinamenti parziali, il costo è invariabile. (Migliore = Medio = Peggior = $\Theta(n^2)$).

9.2 Insertion Sort

Scorre l'array simulando il comportamento di "ordinare una mano di carte", espandendo iterativamente un prefisso già ordinato e inserendo l'elemento i -esimo nella posizione adatta spostando verso destra i valori più alti.

- **Caso Migliore:** Input già ordinato. Nessun elemento viene spostato. Costo $\Theta(n)$.
- **Caso Medio e Peggior (Ordinato al contrario):** L'elemento va spostato fino all'inizio. Costo $\Theta(n^2)$.

9.3 Merge Sort

Applica rigorosamente il *Divide et Impera*.

1. **Divide:** Spezza virtualmente a metà il vettore, ricorsivamente fino ad avere array di lunghezza 1.
2. **Impera/Combina (Merge):** Confronta i primi elementi di due array già ordinati per rimetterli progressivamente all'interno di un vettore di appoggio B ordinato. (Costo operazione Merge: $\Theta(n)$).

L'albero di esecuzione ha profondità $\log n$ e ad ogni livello il costo per ricombinare (cn) copre tutto l'array.

- **Costo Computazionale:** $T(n) = 2T(n/2) + cn = \Theta(n \log n)$ per ogni tipo di input.

Nota pratica: Nonostante Merge Sort abbia una superiorità asintotica su Insertion Sort, i bassi costi delle costanti e dell'overhead per chiamate ricorsive fanno sì che per n molto bassi (es. $n < 64$) Insertion Sort performi empiricamente meglio. Le implementazioni reali usano un mix (es. Timsort).

10 Limiti Teorici dell'Ordinamento e Modelli Alternativi

10.1 Limite Inferiore per l'Ordinamento Basato su Confronti

Qualsiasi algoritmo che ordina un array **effettuando solo confronti** tra coppie di chiavi ha un limite inferiore teorico di $\Omega(n \log n)$. **Dimostrazione:** Per ordinare, un algoritmo deve saper distinguere tutte le possibili permutazioni degli elementi, che sono $n!$. Poiché un confronto ha esito *True* o *False*, i possibili flussi logici formano un albero binario decisionale dove le permutazioni $n!$ risiedono nelle foglie. L'altezza h dell'albero rappresenta il caso pessimo:

$$2^h \geq n! \implies h \geq \log_2(n!)$$

Stimando matematicamente il termine (considerando che i termini della metà superiore superano $n/2$), si ottiene $\log_2(n!) = \Omega(n \log n)$.

10.2 Counting Sort: Superare il limite cambiando Modello

Se si cambia il modello di calcolo e l'ipotesi sui dati di partenza, si può violare il limite di $\Omega(n \log n)$.

Ipotesi: I dati da ordinare non sono reali o stringhe arbitrarie, ma **interi** in un range noto e limitato $[0 \dots k-1]$. **Idea:** Si allocano contatori per ogni possibile valore (array B di dimensione k). Si legge ogni input $A[j]$ e non lo si confronta, ma si va *direttamente* all'indice $B[A[j]]$ e lo si incrementa. Infine si ricostruisce l'array scartando iterativamente i contatori.

- **Costo temporale:** $\Theta(n + k)$.
- **Costo spaziale aggiuntivo:** $\Theta(k)$.

Conseguenze:

- Se $k = \mathcal{O}(n)$ (es. 1 milione di persone con età tra 0 e 120 anni), il costo è incredibilmente lineare e fulmineo: $\Theta(n)$.
- Se $k = \Omega(2^n)$ (intervalli di numeri enormemente grandi rispetto agli elementi totali), diventa inutilizzabile per ragioni di spazio e tempo.

11 Risoluzione delle Equazioni di Ricorrenza

Il costo di un algoritmo ricorsivo dipende dal costo delle chiamate su input di dimensioni ridotte. Questa dipendenza è descritta matematicamente da una **relazione di ricorrenza**. L'obiettivo dell'analisi è trovare una forma chiusa (asintotica) per tale equazione (es. per il Merge Sort, calcolare che $T(n) = \Theta(n \log n)$).

Esistono tre metodi complementari per risolvere le ricorrenze:

1. **Analisi per livelli (Albero delle chiamate):** Rende visibili il numero e il costo delle chiamate livello per livello.

2. **Metodo della sostituzione:** Formula un'ipotesi e la dimostra rigorosamente per induzione matematica.
3. **Teorema dell'esperto (Master Theorem):** Risolve direttamente e in modo meccanico intere famiglie di ricorrenze note.

11.1 1. Analisi per Livelli (Albero delle Chiamate)

L'idea è espandere l'equazione calcolando, per ogni livello dell'albero di ricorsione:

- Il numero di chiamate e la dimensione dei sottoproblemi.
- Il costo non ricorsivo di una singola chiamata (il lavoro locale).
- Il costo complessivo del livello.

La complessità totale dipende dal confronto tra la **crescita del numero di sottoproblemi** e la **diminuzione del costo del lavoro locale**. Osserviamo tre casi tipici aventi 4 sottoproblemi di dimensione $n/2$:

- **Caso 1:** $T(n) = 4T(n/2) + n$
 - Il costo dei livelli cresce scendendo verso il basso.
 - Dominano le foglie (l'ultimo livello costa n^2).
 - Risultato: $\Theta(n^2)$.
- **Caso 2:** $T(n) = 4T(n/2) + n^2$
 - L'aumento delle chiamate compensa esattamente la diminuzione del costo locale.
 - Tutti i livelli costano n^2 , e ci sono $\log n$ livelli.
 - Risultato: $\Theta(n^2 \log n)$.
- **Caso 3:** $T(n) = 4T(n/2) + n^3$
 - Il costo diminuisce scendendo nell'albero.
 - Domina la radice (il primo livello costa n^3).
 - Risultato: $\Theta(n^3)$.

11.2 2. Metodo della Sostituzione (o per Tentativi)

Questo metodo consiste nell'intuire ("indovinare") una possibile soluzione asintotica e dimostrarne la validità tramite **induzione matematica**.

Procedura del Metodo della Sostituzione

1. **Tentativo:** Si ipotizza un limite (es. $T(n) = \mathcal{O}(n)$ significa ipotizzare $T(n) \leq c \cdot n$).
2. **Ipotesi Induttiva:** Si assume che l'ipotesi sia vera per ogni $k < n$ (es. $T(k) \leq c \cdot k$).
3. **Passo Induttivo:** Si sostituisce l'ipotesi induttiva all'interno della ricorrenza di partenza e si dimostra la disequazione per $T(n)$ tramite passaggi algebrici, trovando le costanti valide.
4. **Caso Base:** Si verifica la correttezza della costante sul caso base (es. $T(1)$ o $T(2)$).

Complicazioni comuni e astuzie matematiche:

- **Problemi con i termini di ordine inferiore:** A volte un limite superiore corretto fallisce nel passo induttivo (es. si ottiene $cn + 1 \leq cn$, che è impossibile). In questi casi, bisogna formulare un'ipotesi induttiva *più stretta* sottraendo un termine di grado inferiore: ipotizzare $T(k) \leq ck - b$ (con $b > 0$).
- **Problemi con i casi base:** A causa della presenza di fattori logaritmici, il caso base $n = 1$ potrebbe non essere verificabile (poiché $\log 1 = 0$). Sfruttando le proprietà asintotiche, si può aggirare il problema spostando il caso base a $n = 2$ o $n = 3$, purché la condizione valga per tutti gli $n \geq m$.

11.3 3. Il Teorema dell'Esperto (Master Theorem)

Risolve in modo diretto le ricorrenze basate sulla tecnica *divide-et-impera*.

Teorema dell'Esperto (Versione Ridotta)

Siano $a \geq 1$ e $b > 1$ costanti, $c, d > 0$ e $\beta \geq 0$. Data la ricorrenza:

$$T(n) = \begin{cases} d & n \leq 1 \\ aT(n/b) + cn^\beta & n > 1 \end{cases}$$

Definiamo $\alpha = \log_b a$. Confrontando α e β si ottiene:

- Se $\alpha > \beta \implies T(n) = \Theta(n^\alpha)$
- Se $\alpha = \beta \implies T(n) = \Theta(n^\alpha \log n)$
- Se $\alpha < \beta \implies T(n) = \Theta(n^\beta)$

11.3.1 Versione Estesa e Casi Logaritmici

Se il termine non ricorsivo non è un semplice monomio n^β ma una funzione $f(n)$, si confronta $f(n)$ con n^α :

- Se $f(n) = \mathcal{O}(n^{\alpha-\epsilon})$ per un $\epsilon > 0 \implies \Theta(n^\alpha)$.

- Se $f(n) = \Theta(n^\alpha) \implies \Theta(n^\alpha \log n)$.
- Se $f(n) = \Omega(n^{\alpha+\epsilon})$ e rispetta la condizione di regolarità ($af(n/b) \leq cf(n)$ per un $c < 1$) $\implies \Theta(f(n))$.

Estensione con fattori logaritmici: Se $f(n) = \Theta(n^\beta \log^k n)$ e $\alpha = \beta$:

- Per $k > -1 \implies T(n) = \Theta(n^\alpha \log^{k+1} n)$.
- Per $k = -1 \implies T(n) = \Theta(n^\alpha \log \log n)$.

11.4 Altre Ricorrenze Comuni (Lineari)

Non tutti gli algoritmi dividono l'input in frazioni n/b . Alcuni riducono l'input di una costante (es. $n-1$, $n-2$). Per equazioni del tipo: $T(n) = \sum_{i=1}^h a_i T(n-i) + \Theta(n^\beta)$ (con $a = \sum a_i$):

- Se $a = 1 \implies T(n) = \Theta(n^{\beta+1})$ (es. $T(n) = T(n-1) + n \implies \Theta(n^2)$).
- Se $a \geq 2 \implies T(n) = \mathcal{O}(a^n)$ (es. $T(n) = 2T(n-1) + 1 \implies \mathcal{O}(2^n)$).

12 Ritorno alla Somma Massimale: Analisi Formale

Applichiamo gli strumenti matematici appena definiti per dimostrare le complessità delle 4 versioni dell'algoritmo della somma massimale viste nell'introduzione del corso.

- **Versione 1 (Forza Bruta - $\Theta(n^3)$):** Si contano le esecuzioni del ciclo più interno:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1)$$

Maggiorando le sommatorie si dimostra il limite superiore $T(n) \leq \sum \sum n = \sum n^2 = n^3 = \mathcal{O}(n^3)$. Minorando opportunamente gli indici, si dimostra il limite inferiore $T(n) \geq \frac{n^3}{32} = \Omega(n^3)$.

- **Versione 2 (Ottimizzata - $\Theta(n^2)$):**
Il ciclo più interno viene eseguito $n - i$ volte:

$$T(n) = \sum_{i=0}^{n-1} (n - i) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

- **Versione 3 (Divide-et-Impera - $\Theta(n \log n)$):**
L'algoritmo genera due chiamate ricorsive su metà array e un ciclo lineare per unire le soluzioni[cite: 5]. La ricorrenza è:

$$T(n) = 2T(n/2) + n$$

Applicando il **Master Theorem** (ridotto) con $a = 2, b = 2, \beta = 1 \implies \alpha = \log_2 2 = 1 = \beta$. Rientra nel caso 2 del teorema, quindi $T(n) = \Theta(n \log n)$ [cite: 5].

- **Versione 4 (Programmazione Dinamica - $\Theta(n)$):**
L'algoritmo (Kadane) scansiona l'array con un singolo ciclo `for` da 0 a $n - 1$. È banale dedurre che $T(n) = \Theta(n)$ [cite: 5].

13 Analisi Ammortizzata

Definizione: Analisi Ammortizzata

L'analisi ammortizzata è una tecnica utilizzata per valutare il tempo richiesto per eseguire, **nel caso pessimo, una sequenza di operazioni** su una struttura dati. Se le operazioni più costose sono poco frequenti, il loro costo può essere "spalmato" (ammortizzato) dalle operazioni meno costose e più frequenti.

Differenza cruciale con il Caso Medio:

- *Caso medio:* È un'analisi probabilistica basata su una *singola* operazione e sulla distribuzione dell'input.
- *Analisi ammortizzata:* È un'analisi **deterministica** su *operazioni multiple*, valutando un limite garantito nel caso pessimo dell'intera sequenza.

Esistono tre metodi fondamentali per effettuare un'analisi ammortizzata:

1. **Metodo dell'aggregazione:** Tecnica matematica (calcola la complessità $T(n)$ per n operazioni in sequenza).
2. **Metodo degli accantonamenti:** Tecnica di stampo economico (basata su "crediti" e "debiti").
3. **Metodo del potenziale:** Tecnica di stampo fisico (basata su "l'energia potenziale" del sistema).

13.1 Caso di Studio: Contatore Binario

Consideriamo un contatore binario a k bit implementato come un vettore A di booleani (0/1), dove $A[0]$ è il LSB e $A[k-1]$ è il MSB. L'operazione da analizzare è l'incremento di 1.

```
% Operazione di incremento del contatore
increment(int[] A, int k)
int i = 0
% Finché trovo 1, li azzerò (riporto)
while i < k and A[i] == 1 do
A[i] = 0
i = i + 1
% Se non ho superato i bit disponibili, accendo un bit a 1
if i < k then
A[i] = 1
```

13.1.1 1. Metodo dell'Aggregazione

Si calcola la complessità totale $T(n)$ per eseguire n operazioni e la si divide per n , ottenendo la media nel caso pessimo.

- *Analisi grossolana:* Ogni operazione costa al massimo $\mathcal{O}(k) = \mathcal{O}(\log n)$ bit flip. Quindi n operazioni costano $\mathcal{O}(n \log n)$. Costo ammortizzato: $\mathcal{O}(\log n)$.
- *Analisi stretta (Aggregazione):* Non tutti i bit cambiano ogni volta. $A[0]$ cambia ogni incremento, $A[1]$ ogni 2, $A[2]$ ogni 4, e in generale $A[i]$ cambia ogni 2^i incrementi.

Il costo totale reale è la somma dei bit modificati:

$$T(n) = \sum_{i=0}^{k-1} \lfloor \frac{n}{2^i} \rfloor \leq n \sum_{i=0}^{k-1} \frac{1}{2^i} \leq n \sum_{i=0}^{+\infty} \left(\frac{1}{2}\right)^i = 2n$$

Costo ammortizzato: $\frac{T(n)}{n} \leq \frac{2n}{n} = 2 = \mathcal{O}(1)$.

13.1.2 2. Metodo degli Accantonamenti

Si assegna un **costo ammortizzato fittizio** alle operazioni. Le operazioni meno costose vengono caricate di un costo aggiuntivo detto *credito*, che viene consumato per pagare le operazioni più costose in futuro. L'invariante è che il credito totale cumulato ≥ 0 .

- Costo effettivo per modificare un bit: 1.

- Fissiamo il **costo ammortizzato dell'operazione a 2**.
- Quando capovolgiamo un $0 \rightarrow 1$, paghiamo 1 per l'operazione reale e **accantoniamo** 1 come credito associato a quel bit.
- Quando, per il riporto, dobbiamo azzerare un $1 \rightarrow 0$, il costo reale è 1, ma è **pagato dal credito** accumulato precedentemente su quel bit.
- Poiché in ogni istante il credito è pari al numero di bit a 1 (sempre ≥ 0), il costo totale per n operazioni è limitato da $2n = \mathcal{O}(n)$. Costo ammortizzato: **$\mathcal{O}(1)$** .

13.1.3 3. Metodo del Potenziale

Lo stato del sistema è descritto da una **funzione di potenziale** $\Phi(S)$, che rappresenta il lavoro contabilizzato ma non ancora eseguito. Formula del costo ammortizzato dell' i -esima operazione:

$$a_i = c_i + \Phi(S_i) - \Phi(S_{i-1})$$

(Costo ammortizzato = Costo effettivo + Variazione di potenziale).

Nel contatore binario:

- Poniamo $\Phi(S)$ = numero di bit a 1 presenti nel contatore.
- Durante un incremento incontriamo t bit a 1 che diventano 0, e un bit a 0 che diventa 1.
- **Costo effettivo:** $c_i = 1 + t$ (i t bit azzerati più l'uno acceso).
- **Variazione di potenziale:** Il numero di bit a 1 aumenta di 1 e diminuisce di $t \implies \Phi(S_i) - \Phi(S_{i-1}) = 1 - t$.
- **Costo ammortizzato:** $a_i = (1 + t) + (1 - t) = 2 = \mathcal{O}(1)$.

Poiché $\Phi(S_0) = 0$ e $\Phi(S_n) \geq 0$, la differenza di potenziale globale è non negativa e l'analisi è valida.

14 Vettori Dinamici (Dynamic Arrays)

I vettori dinamici (`ArrayList` in Java, `std::vector` in C++, `list` in Python) allocano una memoria di *capacità* prestabilita. Gli inserimenti in coda (`append`) costano $\mathcal{O}(1)$, finché non si raggiunge la capacità massima. Quando la capacità è satura, occorre un'espansione: si alloca un vettore più grande, si copiano gli elementi vecchi ($\mathcal{O}(n)$) e si dealloca il vecchio.

14.1 Politiche di Espansione

Confrontiamo due metodi comuni per scegliere la nuova dimensione:

14.1.1 1. Incremento Fisso (es. + Costante d)

- Il costo dell'espansione (copia) accade ogni volta che si riempie il blocco d .
- Il costo dell' i -esima `add()` vale i se $i \pmod d = 1$, altrimenti 1.
- Costo effettivo per n operazioni: $T(n) = n + d \sum_{j=1}^{n/d} j \leq n + \frac{(n/d+1)n}{2} = \mathcal{O}(n^2)$.

14.2 Costo ammortizzato: $T(n)/n = \mathcal{O}(n)$

Pessimo approccio.

14.2.1 2. Raddoppiamento (es. $\times 2$)

- Il costo dell'espansione è proporzionale a i solo quando $i = 2^k + 1$.
- Costo effettivo per n operazioni: $T(n) = n + \sum_{j=0}^{\log n} 2^j \leq n + 2^{\log n + 1} - 1 = n + 2n - 1 = \mathcal{O}(n)$.
- Costo ammortizzato: $T(n)/n = \mathcal{O}(1)$. **Approccio standard ed efficiente.**

Reality Check sulle Stringhe (Java/Python)

I vettori dinamici spiegano perché in Java usare `String` (che è immutabile e genera una nuova copia $\mathcal{O}(n^2)$ ad ogni concatenazione) sia letale rispetto a `StringBuffer` (che usa internamente un array raddoppiabile con ammortamento $\mathcal{O}(1)$). Per n elevati (es. concatenare 1 milione di stringhe), `StringBuffer` impiega millisecondi, mentre `String` impiega minuti.

14.3 Cancellazione e Contrazione

Se rimuoviamo elementi dal vettore dinamico, la memoria rimarrà allocata inutilmente. Dobbiamo eseguire una *contrazione* riducendo la dimensione quando il **fattore di carico** $\alpha = \frac{\text{dim}}{\text{capacità}}$ scende sotto una certa soglia.

Il pericolo del Thrashing (Strategia Naïf): Dimezzare il vettore non appena $\alpha = 1/2$ è rischioso. Se la sequenza di operazioni alterna continuamente *Insert*, *Remove*, *Insert*, *Remove* proprio sul bordo della capacità, il sistema farà continue espansioni e contrazioni, producendo un costo ammortizzato lineare $\mathcal{O}(n)$.

Soluzione Corretta: Dobbiamo lasciar decrescere il sistema e dimezzare la memoria solo quando $\alpha = 1/4$. In questo modo, dopo una contrazione, il vettore si ritrova con $\alpha = 1/2$. C'è quindi un "cuscinetto" di $\mathcal{O}(n)$ operazioni economiche prima che sia necessaria un'altra operazione strutturale.

Dimostrazione col Potenziale per la cancellazione:

$$\Phi = \begin{cases} 2 \cdot \dim - \text{capacità} & \text{se } \alpha \geq \frac{1}{2} \\ \frac{\text{capacità}}{2} - \dim & \text{se } \alpha \leq \frac{1}{2} \end{cases}$$

Con questa funzione:

- Subito dopo un resize ($\alpha = 1/2$), $\Phi = 0$.
- Prima di un'espansione ($\alpha = 1$), $\Phi = \dim$ (il potenziale accumulato paga l'espansione).
- Prima di una contrazione ($\alpha = 1/4 \implies \text{capacità} = 4 \cdot \dim$), $\Phi = \frac{4 \cdot \dim}{2} - \dim = \dim$ (il potenziale accumulato paga la contrazione).
- Il costo ammortizzato a_i risulta sempre limitato dalla costante $3 = \mathcal{O}(1)$.

15 Applicazioni Pratiche e Conclusioni

L'analisi ammortizzata e i vettori dinamici trovano implementazione diretta in:

- **Tabelle Hash:** La ristrutturazione (rehashing) di una tabella hash per mantenere basso il fattore di carico t_α si comporta esattamente come i vettori dinamici. Quando t_α supera la soglia (es. 0.75 in Java), la tabella viene raddoppiata. Il caso pessimo della singola operazione è $\mathcal{O}(n)$, ma il **costo ammortizzato** è $\mathcal{O}(1)$.
- **Insiemi disgiunti** (Union-Find con compressione dei cammini).
- **Heap di Fibonacci.**

16 Strutture di Dati Astratte: Definizioni Fondamentali

Definizioni: Dato e Tipo di Dato Astratto

- **Dato:** In un linguaggio di programmazione, è un valore che una variabile può assumere.
- **Tipo di dato astratto (ADT):** Un modello matematico definito da una collezione di valori e un insieme di operazioni ammesse su di essi.
- **Tipi di dato primitivi:** Sono i tipi forniti direttamente dal linguaggio di programmazione (es. `int`, `boolean`) insieme ai loro operatori base (`+`, `-`, `*`, `/`, `%`, `&&`, `||`).

16.1 Specifica vs Implementazione

Nella progettazione del software è fondamentale separare il "cosa" dal "come":

- **Specifica:** Agisce come un "manuale d'uso"; definisce le operazioni nascondendo i dettagli implementativi all'utilizzatore.
- **Implementazione:** È la realizzazione vera e propria della struttura in memoria e del codice degli algoritmi.
- *Esempio pratico:* Una Pila (Stack) può essere specificata in modo astratto, ma implementata concretamente tramite vettori oppure tramite puntatori (nodi collegati).

16.2 Caratteristiche delle Strutture Dati

Le strutture di dati sono collezioni caratterizzate più dall'organizzazione interna che dal tipo di dati contenuti. Possono essere classificate in base a:

- **Linearità:** Lineari (esiste un concetto di sequenza e successione) vs Non lineari (alberi, grafi).
- **Dinamicità:** Statiche (dimensione fissa) vs Dinamiche (variazione di dimensione e contenuto a runtime).

- **Omogeneità:** Omogenee (dati tutti dello stesso tipo) vs Disomogenee.

17 Tipi di Dato Astratto Principali

17.1 1. Sequenza

Una struttura dati dinamica e lineare che rappresenta una collezione ordinata di valori.

- L'ordine all'interno della sequenza è fondamentale.
- Un valore può comparire più di una volta (sono ammessi duplicati).
- Date le posizioni (da pos_1 a pos_n , più le fittizie pos_0 e pos_{n+1}), è possibile accedere alla testa, alla coda o sequenzialmente agli elementi.

```
% Pseudocodice: Specifica della Sequenza
boolean isEmpty()           % Verifica se vuota
Pos head()                  % Posizione primo elemento
Pos tail()                  % Posizione ultimo elemento
Pos next(Pos p)             % Posizione successore
Pos prev(Pos p)             % Posizione predecessore
Pos insert(Pos p, item v)   % Inserisce v in posizione p
Pos remove(Pos p)           % Rimuove l'elemento in p
item read(Pos p)            % Legge il valore in p
```

17.2 2. Insiemi (Set)

Una struttura dati dinamica e non lineare che memorizza una collezione **non ordinata** e **senza duplicati**.

- L'ordinamento, se presente, è derivato unicamente dalla relazione d'ordine definita sul tipo degli elementi stessi.
- Supporta operazioni matematiche insiemistiche: unione, intersezione e differenza.

```
% Pseudocodice: Specifica dell'Insieme
int size()                  % Cardinalità
boolean contains(item x)    % Verifica appartenenza
insert(item x)              % Inserisce (se non presente)
remove(item x)              % Rimuove (se presente)
SET union(SET A, SET B)     % Unione insiemistica
SET intersection(SET A, SET B) % Intersezione insiemistica
SET difference(SET A, SET B) % Differenza insiemistica
```

17.3 3. Dizionari (Mappe / Array Associativi)

Rappresenta il concetto matematico di relazione univoca $R : D \rightarrow C$, nota come associazione **chiave-valore**.

- L'insieme D è il dominio (le **chiavi**, che devono essere univoche).
- L'insieme C è il codominio (i **valori**).

```
% Pseudocodice: Specifica del Dizionario
item lookup(item k)         % Restituisce il valore associato a k (o nil)
insert(item k, item v)      % Associa v a k (sovrascrive se k già esiste)
remove(item k)              % Rimuove l'associazione per la chiave k
```

17.4 4. Alberi e Grafi

- **Alberi ordinati:** Insieme finito di nodi con un nodo designato come **radice**. I restanti nodi sono partizionati in insiemi disgiunti che formano a loro volta alberi (sottoalberi).
- **Grafi:** Struttura composta da un insieme di nodi (o vertici) e un insieme di coppie di nodi dette **archi** (orientati o non orientati).
- Tutte le operazioni su queste due strutture ruotano attorno alla possibilità di effettuare **visite** per esplorare i nodi.

18 Implementazione delle Strutture Dati Elementari

18.1 Lista (Linked List)

È un'implementazione classica della sequenza. È composta da una serie di **nodi** contenenti dati e puntatori all'elemento successivo (e/o precedente).

- La contiguità logica nella lista **non** implica contiguità fisica in memoria.
- Tutte le operazioni base (inserimento, rimozione in posizione nota) hanno costo $\mathcal{O}(1)$.
- **Varianti:** Monodirezionale, Bidirezionale, Circolare, Con sentinella / Senza sentinella.

```
// Snippet Java: Inserimento in Lista Bidirezionale (senza sentinella)
public Pos insert(Pos pos, Object v) {
    Pos t = new Pos(v);
    if (head == null) {
        head = tail = t; // Lista vuota
    } else if (pos == null) {
        t.pred = tail; // Inserimento in coda
        tail.succ = t;
        tail = t;
    } else {
        t.pred = pos.pred; // Inserimento prima di 'pos'
        if (t.pred != null) t.pred.succ = t;
        else head = t;
        t.succ = pos;
        pos.pred = t;
    }
    return t;
}
```

18.2 Pila (Stack)

Struttura dati dinamica e lineare basata sul principio **LIFO** (Last-In, First-Out): l'elemento rimosso è sempre quello rimasto per meno tempo nell'insieme.

- **Operazioni base:** push (inserisce in cima), pop (estrae dalla cima), top (legge la cima senza estrarre).
- **Utilizzi pratici:** Gestione dei record di attivazione (stack memory delle funzioni), linguaggi stack-oriented come la JVM (Java bytecode) o Postscript.
- **Implementazioni:** Può essere realizzata tramite liste (puntatore alla testa) o tramite vettori.

```
% Pseudocodice: Pila basata su vettore (Array)
push(item v)
precondition: size < capacity
A[size] = v
```

```

size = size + 1

item pop()
precondition: size > 0
size = size - 1
return A[size]

```

18.3 Coda (Queue)

Struttura dati dinamica e lineare basata sul principio **FIFO** (First-In, First-Out): l'elemento rimosso è quello che per più tempo è rimasto nell'insieme.

- **Operazioni base:** `enqueue` (inserisce in fondo), `dequeue` (estrae dalla testa), `top` (legge la testa).
- **Utilizzi pratici:** Nei sistemi operativi per gestire le code di attesa dei processi, in quanto la politica FIFO garantisce *fairness* (equità).
- **Implementazioni:** Liste (con due puntatori `head` e `tail`) oppure **Vettori Circolari**.

La Coda basata su Vettore Circolare

Per evitare lo scorrimento degli elementi all'interno di un vettore dopo ogni `dequeue`, si usa un vettore in cui la coda "gira" ricollegandosi all'inizio. La circolarità è implementata matematicamente tramite l'operatore **modulo** (%).

```

% Pseudocodice: Coda basata su vettore circolare
enqueue(item v)
precondition: size < capacity
% Calcola l'indice di inserimento con il modulo per la circolarità
A[(head + size) mod capacity] = v
size = size + 1

item dequeue()
precondition: size > 0
item t = A[head]
% Avanza la testa circolarmente
head = (head + 1) mod capacity
size = size - 1
return t

```

19 Alberi

19.1 Introduzione e Definizioni Formali

Gli alberi sono strutture dati non lineari utilizzate per rappresentare dati con relazioni gerarchiche (es. tassonomie, file system, alberi DOM HTML).

Definizioni di Albero Radicato

Definizione 1 (strutturale): Un albero è un insieme di nodi e di archi orientati che connettono coppie di nodi con le seguenti proprietà:

- Un nodo è designato come **radice**.
- Ogni nodo (eccetto la radice) ha esattamente **un arco entrante**.
- Esiste un **cammino unico** dalla radice a ogni nodo.
- L'albero è **connesso**.

Definizione 2 (ricorsiva): Un albero è dato da un insieme vuoto, oppure da un nodo radice collegato con archi orientati alle radici di zero o più sottoalberi disgiunti.

19.1.1 Terminologia e Metriche

- **Padre / Figli / Fratelli (Siblings):** Nodi con lo stesso padre sono fratelli.
- **Foglie (Leaves):** Nodi privi di figli.
- **Nodi interni:** Nodi che hanno almeno un figlio (non sono foglie).
- **Profondità (Depth):** La lunghezza del cammino semplice (misurato in numero di archi) dalla radice al nodo.
- **Livello (Level):** L'insieme dei nodi che si trovano alla stessa profondità.
- **Altezza (Height):** La profondità massima raggiunta dalle foglie dell'albero.

19.2 Alberi Binari

Un albero binario è un albero radicato in cui ogni nodo ha **al massimo due figli**, identificati esplicitamente come **figlio sinistro** e **figlio destro**. *Nota cruciale:* Due alberi con gli stessi nodi sono considerati **distinti** se un nodo è figlio sinistro in un albero ma figlio destro nell'altro.

19.2.1 Implementazione in Memoria

Per memorizzare un albero binario si utilizzano nodi contenenti tre reference (puntatori):

- **parent:** reference al nodo padre (oppure `nil` se radice).
- **left:** reference al figlio sinistro.
- **right:** reference al figlio destro.

```
% Operazioni base di inserimento e distruzione per alberi binari
insertLeft(TREE T)
if left == nil then
T.parent = this
left = T

deleteLeft()
if left != nil then
left.deleteLeft()    % Chiamata ricorsiva
left.deleteRight()   % Chiamata ricorsiva
delete left
left = nil
```

19.3 Visite di Alberi (Traversals)

Una visita (o ricerca) è una strategia sistematica per analizzare tutti i nodi di un albero. **Costo computazionale:** $\Theta(n)$, poiché ogni nodo viene visitato esattamente una volta.

19.3.1 1. Visita in Profondità (Depth-First Search - DFS)

Esplora l'albero visitando ricorsivamente i sottoalberi scendendo il più possibile in profondità. Tipicamente si appoggia a uno **Stack** (implicito tramite ricorsione o esplicito). Si divide in tre varianti a seconda di quando viene processata la radice:

- **Pre-Order (Pre-visita):** Stampa radice, visita sx, visita dx.
- **In-Order (In-visita):** Visita sx, stampa radice, visita dx.
- **Post-Order (Post-visita):** Visita sx, visita dx, stampa radice.

Esempi di applicazione della DFS:

```

% 1. Contare i nodi dell'albero (Utilizza la Post-visita)
int count(TREE T)
if T == nil then return 0
else
Cl = count(T.left())
Cr = count(T.right())
return Cl + Cr + 1

% 2. Stampare espressioni matematiche (Utilizza l'In-visita)
int printExp(TREE T)
if T.left() == nil and T.right() == nil then
print T.read()
else
print "("
printExp(T.left())
print T.read()
printExp(T.right())
print ")"

```

19.3.2 2. Visita in Ampiezza (Breadth-First Search - BFS)

Visita l'albero procedendo per **livelli**: parte dalla radice e visita tutti i nodi di un livello prima di scendere al successivo. Richiede l'uso di una struttura dati ausiliaria di tipo **Queue** (Coda).

19.4 Alberi Generici

In un albero generico, i nodi possono avere un numero arbitrario di figli. La specifica si adatta per gestire le liste di figli. Operazioni base principali:

- `leftmostChild()`: Restituisce il primo figlio (o `nil` se foglia).
- `rightSibling()`: Restituisce il prossimo fratello (o `nil` se assente).

19.4.1 Visite per Alberi Generici

Il concetto di *In-Order* perde di significato, ma DFS (Pre e Post) e BFS rimangono validi.

```
% DFS su Albero Generico
dfs(TREE t)
if t != nil then
print t % Pre-order
TREE u = t.leftmostChild()
while u != nil do
dfs(u)
u = u.rightSibling()
print t % Post-order
```

```
% BFS su Albero Generico
bfs(TREE t)
QUEUE Q = Queue()
Q.enqueue(t)
while not Q.isEmpty() do
TREE u = Q.dequeue()
print u
u = u.leftmostChild()
while u != nil do
Q.enqueue(u)
u = u.rightSibling()
```

19.4.2 Implementazioni in Memoria per Alberi Generici

A seconda del numero massimo/medio di figli, si possono adottare tre strategie di memorizzazione:

1. **Vettore dei figli:** Ogni nodo ha un reference al padre e un array contenente i puntatori ai figli. Può comportare un notevole spreco di spazio se la maggior parte dei nodi ha pochi figli.
2. **Primo figlio, prossimo fratello (First-Child / Next-Sibling):** È l'implementazione più diffusa ed efficiente in spazio per alberi generici (modella l'albero come una lista concatenata di fratelli). I campi memorizzati in ogni nodo sono:
 - `parent`: reference al padre.
 - `child`: reference al primo figlio.
 - `sibling`: reference al prossimo fratello.

```
% Esempio inserimento: "Primo figlio, prossimo fratello"
insertChild(TREE t)
t.parent = self
t.sibling = child
child = t      % t diventa il nuovo primo figlio
```

3. **Vettore dei padri:** L'albero è rappresentato interamente da un singolo vettore. Ogni elemento in posizione i contiene il **valore** del nodo e l'**indice** della posizione di suo padre nel vettore stesso[cite: 8]. Utile per specifiche strutture (come gli insiemi disgiunti)[cite: 8].

20 Alberi Binari di Ricerca (ABR)

Un **Albero Binario di Ricerca (ABR)** implementa l'astrazione del *Dizionario* (ricerca, inserimento e cancellazione di associazioni chiave-valore) unendo la flessibilità strutturale degli alberi con la logica della ricerca dicotomica.

Proprietà Fondamentale degli ABR

In un albero binario di ricerca, per ogni nodo u :

1. Le chiavi contenute nei nodi del sottoalbero **sinistro** di u sono **minori** di $u.key$.
2. Le chiavi contenute nei nodi del sottoalbero **destro** di u sono **maggiori** di $u.key$.

Queste due proprietà permettono di scartare interi sottoalberi durante la ricerca, riducendo drasticamente i tempi operativi rispetto a una struttura lineare.

20.1 Struttura del Nodo

Ogni nodo di un ABR mantiene le informazioni strutturali e i dati veri e propri:

- **parent, left, right:** Reference al padre, al figlio sinistro e al figlio destro.
- **key, value:** Chiave (appartenente a un insieme totalmente ordinato) e valore associato.

20.2 Operazioni di Base

20.2.1 Ricerca (Lookup)

La ricerca parte dalla radice e scende a sinistra o a destra confrontando la chiave cercata con la chiave del nodo corrente.

```
% Implementazione Ricorsiva
TREE lookupNode(TREE T, item k)
if T == nil or T.key == k then
return T
```

```
return lookupNode( iif(k < T.key, T.left, T.right), k )
```

```
% Implementazione Iterativa
TREE lookupNode(TREE T, item k)
TREE u = T
while u != nil and u.key != k do
  if k < u.key then u = u.left
  else u = u.right
return u
```

20.2.2 Ricerca del Minimo e del Massimo

Grazie alle proprietà dell'ABR, il minimo si trova percorrendo sempre i rami sinistri, mentre il massimo percorrendo i rami destri.

- **Minimo:** while `u.left != nil` do `u = u.left`.
- **Massimo:** while `u.right != nil` do `u = u.right`.

20.2.3 Successore e Predecessore

Il **successore** di un nodo u è il nodo con la chiave immediatamente superiore (il più piccolo dei maggiori). Si presentano due casi:

1. **Il nodo ha un figlio destro:** Il successore è il *minimo* del sottoalbero destro.
2. **Il nodo NON ha un figlio destro:** Risalendo lungo l'albero tramite i padri, il successore è il primo "avo" v tale per cui u risiede nel sottoalbero sinistro di v .

Nota: La ricerca del **predecessore** è speculare (massimo del sottoalbero sinistro, oppure primo avo a destra).

20.3 Inserimento e Cancellazione

20.3.1 Inserimento

La ricerca del punto di inserimento avviene tramite le stesse regole della funzione `lookup`. Si inserisce il nuovo nodo come foglia. Se la chiave è già presente, se ne aggiorna il valore.

20.3.2 Cancellazione

La rimozione di un nodo u è complessa e prevede tre casi distinti:

- **Caso 1 (u è una foglia):** Il nodo viene semplicemente eliminato staccandolo dal padre.
- **Caso 2 (u ha un solo figlio):** Si aggira il nodo u attaccando direttamente il suo unico figlio all'ex-padre di u (*short-cut*).
- **Caso 3 (u ha due figli):**
 1. Si individua il **successore** s di u (che sicuramente non avrà un figlio sinistro).
 2. Si stacca s dal suo padre (ricadendo nel Caso 1 o Caso 2 per s).
 3. Si copia il contenuto di s in u e si rimuove il nodo originario di s .

20.4 Costo Computazionale degli ABR

Tutte le operazioni descritte (ricerca, min/max, succ/pred, inserimento, cancellazione) avvengono percorrendo un **cammino semplice** dalla radice verso una foglia. Il tempo operativo dipende quindi dall'**altezza** h dell'albero:

$$T(n) = \mathcal{O}(h)$$

- **Caso Ottimo:** Albero perfettamente bilanciato $\implies h = \mathcal{O}(\log n)$.
- **Caso Pessimo:** Dati inseriti in modo ordinato (l'albero degenera in una lista) $\implies h = \mathcal{O}(n)$.

21 Alberi Binari di Ricerca Bilanciati: Alberi Red-Black

Poiché nel caso pessimo un normale ABR degenera a $\mathcal{O}(n)$, nella pratica informatica moderna (es. `TreeMap` in Java, `std::map` in C++, `rbtree` nel kernel Linux) si impiegano **alberi bilanciati**. Un albero si dice bilanciato quando il suo **fattore di bilanciamento** (la massima differenza di altezza fra i sottoalberi di ogni nodo) è controllato (es. negli alberi AVL è al massimo 1).

Definizione e Regole degli Alberi Red-Black (RB)

In un albero Red-Black ogni nodo ha un colore (**Rosso** o **Nero**) e le foglie reali contengono i dati, mentre i rami "vuoti" puntano a speciali nodi sentinella **Nil**. Devono valere quattro rigorosi vincoli strutturali:

1. **La radice è Nera.**
2. **Tutte le foglie (nodi Nil) sono Nere.**
3. **Entrambi i figli di un nodo Rosso sono Neri** (Non ci possono essere due nodi rossi consecutivi su un cammino).
4. **Altezza Nera Costante:** Ogni cammino semplice da un nodo u a una qualsiasi foglia del suo sottoalbero contiene lo **stesso numero di nodi Neri**.

21.1 Altezza Nera e Teoremi

L'**Altezza Nera** $bh(v)$ di un nodo v è il numero di nodi neri lungo ogni cammino da v (escluso) fino alle foglie (incluse).

Teoremi fondamentali che garantiscono le prestazioni degli Alberi RB:

- **Numero nodi interni:** Un sottoalbero di radice u contiene almeno $n \geq 2^{bh(u)} - 1$ nodi interni.
- **Proporzione nodi neri:** Almeno la metà dei nodi in un percorso radice-foglia deve essere nera (per il vincolo 3, alternanza Rosso-Nero).
- **Sbilanciamento limitato:** Dati due cammini qualsiasi dalla radice alle foglie, uno non può mai essere più del doppio dell'altro.
- **Altezza Massima:** L'altezza massima h di un albero RB con n nodi interni è al più $2\log(n+1)$. *Dimostrazione:* $n \geq 2^{bh(r)} - 1 \implies n \geq 2^{h/2} - 1 \implies n+1 \geq 2^{h/2} \implies \log(n+1) \geq h/2 \implies h \leq 2\log(n+1)$.

Essendo l'altezza sempre vincolata logaritmicamente, **tutte le operazioni costano** $\mathcal{O}(\log n)$.

21.2 Meccanismi di Ribilanciamento: Le Rotazioni

Quando inserimenti o cancellazioni violano le regole RB, l'albero si ribilancia usando cambi di colore e **rotazioni**.

- **Rotazione a Sinistra su x :** y (figlio destro di x) diventa il nuovo padre. x diventa il figlio sinistro di y . Il sottoalbero sinistro di y diventa il figlio destro di x . (Le operazioni mantengono intatto l'ordinamento ABR).
- **Rotazione a Destra:** Procedura speculare.

21.3 Inserimento negli Alberi Red-Black

Il nuovo nodo viene inserito come foglia (con due figli Nil) e **colorato di Rosso**. Questo può violare solo il vincolo 1 (se è la radice) o il vincolo 3 (se il padre è rosso). Sia t il nodo rosso inserito, p il padre, n il nonno e z lo zio. La procedura `balanceInsert` distingue 5 casi (e 5 speculari):

- **Caso 1:** t è radice $\implies$ Colora t di **Nero**.
- **Caso 2:** p è **Nero** $\implies$ Nessun vincolo violato, operazione terminata.
- **Caso 3 (p Rosso, z Rosso):** Colora p e z di **Nero**, e n di **Rosso**. Il problema viene "spinto" in alto verso la radice ($t \leftarrow n$).
- **Caso 4 (p Rosso, z Nero, e nodo t è figlio "interno", es. t figlio dx di p , e p figlio sx di n):** Si applica una rotazione su p per trasformarlo in un allineamento "esterno" (Caso 5).
- **Caso 5 (p Rosso, z Nero, e nodo t è figlio "esterno"):** Si colora p di **Nero**, n di **Rosso**, e si esegue una rotazione su n . I vincoli sono ripristinati e l'algoritmo termina.

Complessità di inserimento: $\mathcal{O}(\log n)$ discese e risalite; massimo 2 rotazioni (costo $\mathcal{O}(1)$).

21.4 Cancellazione negli Alberi Red-Black

Costruita sull'algoritmo standard di cancellazione ABR. Le violazioni avvengono **solo se il nodo rimosso era Nero** (perché diminuisce l'altezza nera di quel cammino, violando il vincolo 4). La procedura `balanceDelete` risolve lo squilibrio propagando la mancanza di "nero" verso l'alto finché non può compensarla con un nodo rosso o fino a raggiungere la radice. Richiede al massimo $\mathcal{O}(\log n)$ cambi di colore e al più 3 rotazioni in tempo $\mathcal{O}(1)$.

22 Hashing e Dizionari

22.1 Motivazioni e Implementazione Ideale

Un **dizionario** è una struttura dati utilizzata per memorizzare insiemi dinamici di coppie (chiave, valore), indicizzate in base alla chiave. Le implementazioni classiche (Vettori ordinati/non ordinati, Liste, Alberi Red-Black) offrono tempi di esecuzione per la ricerca, l'inserimento e la cancellazione che variano da $\mathcal{O}(n)$ a $\mathcal{O}(\log n)$. L'obiettivo dell'hashing è raggiungere un'implementazione ideale in cui tutte le operazioni fondamentali (`insert`, `lookup`, `remove`) abbiano un costo temporale pari a $\mathcal{O}(1)$.

Definizioni Base: Tabella Hash

Data un'insieme di possibili chiavi (l'insieme universo $\mathcal{U}$ di dimensione u), si memorizzano le coppie in un vettore di dimensione m ($m \ll u$), detto **tabella hash**.

- **Funzione Hash:** È una funzione matematica $H : \mathcal{U} \rightarrow \{0, 1, \dots, m - 1\}$ che mappa una chiave in un indice del vettore.
- La coppia (k, v) viene memorizzata esattamente nella posizione $H(k)$.

22.1.1 Tabelle ad Accesso Diretto e Collisioni

Se l'insieme universo $\mathcal{U}$ è molto piccolo e denso (es. i giorni dell'anno da 1 a 366), si può usare una **tabella ad accesso diretto** con funzione identità $H(k) = k$ e $m = u$. Tuttavia, nella realtà lo spazio delle chiavi è enorme e sparso. Riducendo m , si verifica il problema delle **collisioni**: due o più chiavi diverse ottengono lo stesso indice hash ($H(k_1) = H(k_2)$).

23 Progettazione delle Funzioni Hash

Una **funzione hash perfetta** (iniettiva, zero collisioni) è impraticabile da ottenere per domini sparsi. L'obiettivo è quindi **minimizzare le collisioni** distribuendo uniformemente le chiavi nella tabella.

Uniformità Semplice

Sia $P(k)$ la probabilità che la chiave k sia inserita, e $Q(i)$ la probabilità che finisca nella cella i . Una funzione hash gode di **uniformità semplice** se ogni indice è equiprobabile:

$$\forall i \in [0, \dots, m-1] : Q(i) = \frac{1}{m}$$

23.1 Metodi Comuni per Funzioni Hash Semplici

Generalmente si assume che le chiavi testuali o complesse vengano prima tradotte in valori numerici interi (es. convertendo e concatenando i valori ASCII dei caratteri in binario).

- **Estrazione e XOR:** Si estraggono p bit specifici dalla chiave, oppure si fondono gruppi di bit tramite operatore XOR. *Svantaggi:* Possono generare collisioni sistemiche (es. lo XOR mappa anagrammi nello stesso indice).

- **Metodo della Divisione:**

$$H(k) = k \bmod m$$

Regola d'oro: m deve essere dispari, preferibilmente un **numero primo** distante da potenze di 2 e 10. Valori come $m = 2^p$ sono pessimi perché l'hash dipenderebbe solo dai p bit meno significativi della chiave.

- **Metodo della Moltiplicazione (Knuth):**

$$H(k) = \lfloor m \cdot (C \cdot k - \lfloor C \cdot k \rfloor) \rfloor$$

Vantaggi: m può essere qualsiasi (spesso si sceglie una potenza di 2, $m = 2^p$, per efficienza hardware). La costante C deve essere $0 < C < 1$; un valore suggerito è legato alla sezione aurea $C \approx \frac{\sqrt{5}-1}{2}$.

Nota pratica (Reality Check): I metodi classici spesso non superano test statistici moderni (effetto valanga/avalanche effect). Nelle implementazioni reali si usano funzioni hash non-crittografiche estremamente veloci (es. *CityHash*, *FarmHash*, *MurmurHash*) o crittografiche (es. *SHA-1*) se è richiesta sicurezza.

24 Gestione delle Collisioni

Esistono due macro-famiglie per gestire il caso in cui due chiavi finiscano nello stesso slot: **Liste di Trabocco** e **Indirizzamento Aperto**.

24.1 1. Liste/Vettori di Trabocco (Chaining)

Le chiavi con lo stesso indice hash vengono memorizzate in una struttura dinamica esterna (lista monodirezionale o vettore) allocata nello slot $H(k)$.

- **Fattore di carico:** Definito come $\alpha = \frac{n}{m}$ (numero di elementi totali fratto capacità della tabella). Rappresenta la lunghezza media delle liste.
- **Operazione di insert:** Costo $\Theta(1)$ (inserimento in testa alla lista).
- **Operazioni lookup e remove:** Richiedono la scansione della lista. Costo $\Theta(n)$ nel caso pessimo (tutte le chiavi nella stessa lista).
- **Analisi del caso medio:** Una ricerca con *insuccesso* esamina in media $\Theta(1) + \alpha$ nodi. Una ricerca con *successo* esamina in media $\Theta(1) + \frac{\alpha}{2}$ nodi. Se $n = \mathcal{O}(m)$, allora $\alpha = \mathcal{O}(1)$ e le operazioni costano in media $\mathcal{O}(1)$.

24.2 2. Indirizzamento Aperto (Open Addressing)

Nessuna struttura esterna: tutte le chiavi risiedono **direttamente nella tabella hash**. Il fattore di carico α è strettamente limitato a $\alpha \leq 1$. Se lo slot $H(k)$ è occupato, si ispezionano posizioni alternative basandosi su un **numero di ispezione** i . La funzione hash diventa:

$$H : \mathcal{U} \times [0 \dots m - 1] \rightarrow [0 \dots m - 1]$$

La **sequenza di ispezione** per una chiave deve generare una permutazione completa degli indici $[0 \dots m - 1]$ per garantire di poter esaminare tutta la tabella senza loop infiniti.

24.2.1 Tecniche di Ispezione

1. **Ispezione Lineare:** $H(k, i) = (H_0(k) + \delta \cdot i) \bmod m$.
 - *Svantaggio:* Soffre di **agglomerazione primaria (primary clustering)**. Sequenze continue di slot occupati tendono ad allungarsi, poiché uno slot vuoto preceduto da i slot pieni ha un'alta probabilità $(\frac{i+1}{m})$ di essere riempito al prossimo inserimento, degradando le prestazioni.
2. **Ispezione Quadratica:** $H(k, i) = (H_0(k) + \delta \cdot i^2) \bmod m$.
 - *Svantaggio:* Soffre di **agglomerazione secondaria** (chiavi con lo stesso hash iniziale generano identiche sequenze di ispezione alternative).
3. **Doppio Hashing:** $H(k, i) = (H_0(k) + H_\delta(k) \cdot i) \bmod m$.
 - Usa due funzioni hash indipendenti. Genera m^2 sequenze di ispezione distinte, avvicinandosi all'ideale dell'*hashing uniforme*.

- *Vincolo matematico:* Per generare una permutazione completa, $H_\delta(k)$ deve essere primo rispetto ad m . (Es: se m è potenza di 2, $H_\delta(k)$ deve essere dispari; se m è primo, $H_\delta(k) < m$).

24.2.2 La Cancellazione nell'Indirizzamento Aperto

La `remove()` è problematica: **non si può sovrascrivere la cella con `nil`**. Se lo si facesse, si interromperebbe irrimediabilmente la sequenza di ispezione per chiavi inserite successivamente che avevano trovato "traffico" in quello slot.

- **Soluzione:** Sostituire il valore con un marcatore speciale **deleted** (spesso chiamato *tombstone*).
- Durante la `lookup`, i marcatori **deleted** vengono ignorati (trattati come slot pieni) e la ricerca prosegue.
- Durante la `insert`, i marcatori **deleted** vengono trattati come slot vuoti e possono essere sovrascritti.
- *Svantaggio:* L'uso massiccio di cancellazioni degrada i tempi di ricerca svincolandoli dal fattore α . Per tabelle soggette a molte rimozioni, si preferiscono le liste di trabocco.

25 Reality Check e Considerazioni Pratiche

25.1 Implementazioni nel Mondo Reale

A causa della facilità di implementazione e della miglior tolleranza alle cancellazioni e ad alti fattori di carico, la soluzione più adottata nell'industria è il **Chaining (Liste di Trabocco)**.

- **Java 7 (HashMap):** Liste di trabocco basate su `LinkedList`. Overhead di memoria e caso pessimo $\mathcal{O}(n)$.
- **Java 8+ (HashMap):** Usa liste di trabocco, ma quando una singola lista supera una certa soglia, viene convertita automaticamente in un **Albero Red-Black**, riducendo il caso pessimo a $\mathcal{O}(\log n)$.
- **Python (dict):** Utilizza invece l'**indirizzamento aperto** con un fattore di ridimensionamento conservativo ($t_\alpha = 0.66$).

% Regole di base in Java per la funzione `hashCode()`

1. Restituire un intero coerente durante la stessa esecuzione.

2. Se `a.equals(b)` è true, `a.hashCode()` DEVE essere uguale a `b.hashCode()`.

3. (Raccomandato) Se `a.equals(b)` è false, gli hash dovrebbero essere diversi per minimizzare le collisioni.

% REGOLA AUREA: Fare sempre l'override di `hashCode()` se si fa l'override di `equals()`!

25.2 Limiti dell'Hashing

Nonostante l'efficienza $\mathcal{O}(1)$ nel caso medio, le tabelle hash presentano alcune criticità intrinseche:

1. **Mancanza di ordinamento:** Impossibile scorrere le chiavi in ordine (le operazioni min/max costano $\mathcal{O}(m)$).
2. **Scarsa Località spaziale (Locality of Reference):** Gli accessi alla memoria sembrano casuali, causando frequenti *cache miss* a livello hardware.

26 Insiemi e Dizionari: Riassunto delle Implementazioni

Definizioni Fondamentali

- **Insieme:** Una collezione di oggetti. Può essere implementato tramite diverse strutture dati viste finora, ognuna con specifici vantaggi e svantaggi.
- **Dizionario:** Una struttura dati dedicata alla memorizzazione di associazioni chiave-valore.

26.1 Insiemi realizzati con Vettori Booleani

L'insieme è rappresentato considerando un universo di interi da 0 a $m - 1$. La collezione di oggetti viene memorizzata utilizzando un vettore booleano di m elementi.

- **Vantaggi:** Implementazione notevolmente semplice; è estremamente efficiente ($\mathcal{O}(1)$) verificare se un elemento appartiene all'insieme.
- **Svantaggi:** La memoria occupata è sempre $\mathcal{O}(m)$, totalmente indipendente dalle dimensioni effettive dell'insieme (ovvero dal numero di elementi realmente contenuti). Alcune operazioni, come l'iterazione, risultano inefficienti e costano $\mathcal{O}(m)$.

% Pseudocodice: Operazioni base con vettore booleano

```
SET Set(int m)
SET t = new SET
t.size = 0
t.dim = m
t.V = new boolean[0 ... m - 1] = {false}
return t
```

```
boolean contains(int x)
if 0 <= x <= dim - 1 then return V[x]
else return false
```

```
insert(int x)
if 0 <= x <= dim - 1 then
if not V[x] then
size = size + 1
V[x] = true
```

% Pseudocodice: Operazioni insiemistiche con vettore booleano (Costo $\mathcal{O}(m)$)

```
SET union(SET A, SET B)
int newsize = max(A.dim, B.dim)
SET C = Set(newsize)
for i = 0 to A.dim - 1 do
if A.contains(i) then C.insert(i)
for i = 0 to B.dim - 1 do
if B.contains(i) then C.insert(i)
return C
```

```
SET intersection(SET A, SET B)
int newsize = min(A.dim, B.dim)
SET C = Set(newsize)
for i = 0 to min(A.dim, B.dim) - 1 do
if A.contains(i) and B.contains(i) then C.insert(i)
return C
```

Implementazioni Reali:

- **Java:** Classe `java.util.BitSet`. I metodi principali includono `and` (Intersection), `or` (Union), `cardinality` (Set size), `clear` (Remove), `set` (Insert), `get` (Contains).

- **C++ (STL):** `std::bitset` (dimensione fissata a tempo di compilazione tramite template) e `std::vector` (specializzazione per ottimizzare la memoria, con dimensione dinamica).

26.2 Insiemi realizzati con Liste o Vettori Non Ordinati

- Le operazioni di ricerca (lookup), inserimento controllato e cancellazione costano $\mathcal{O}(n)$.
- L'inserimento puro (assumendo a priori l'assenza dell'elemento) costa $\mathcal{O}(1)$.
- Le operazioni insiemistiche (unione, intersezione, differenza) costano $\mathcal{O}(nm)$ poiché per ogni elemento del primo insieme bisogna scorrere il secondo.

```
% Pseudocodice: Differenza con liste non ordinate
SET difference(SET A, SET B)
SET C = Set()
foreach s in A do
if not B.contains(s) then
C.insert(s)
return C
```

26.3 Insiemi realizzati con Liste o Vettori Ordinati

Mantenere l'ordinamento ottimizza alcune operazioni asintotiche:

- **Ricerca:** Costa $\mathcal{O}(n)$ per le liste (scorrimento sequenziale), ma scende a $\mathcal{O}(\log n)$ per i vettori ordinati grazie alla ricerca dicotomica.
- **Inserimento e cancellazione:** Costano sempre $\mathcal{O}(n)$ (necessità di far scorrere gli elementi per fare spazio o riempire i buchi).
- **Unione, intersezione e differenza:** Scendono a $\mathcal{O}(n)$ scorrendo simultaneamente entrambe le strutture (simile alla procedura di Merge).

```
% Pseudocodice: Intersezione efficiente  $\mathcal{O}(n)$  con Liste Ordinate
LIST intersection(LIST A, LIST B)
LIST C = Set()
Pos pos_a = A.head()
Pos pos_b = B.head()
while not A.finished(pos_a) and not B.finished(pos_b) do
if A.read(pos_a) == B.read(pos_b) then
C.insert(C.tail(), A.read(pos_a))
pos_a = A.next(pos_a)
pos_b = B.next(pos_b)
else if A.read(pos_a) < B.read(pos_b) then
pos_a = A.next(pos_a)
else
pos_b = B.next(pos_b)
return C
```

26.4 Insiemi tramite Strutture Dati Complesse

Per performance ottimali si ricorre ad alberi bilanciati o tabelle hash.

- **Alberi Bilanciati:** Garantiscono $\mathcal{O}(\log n)$ per ricerca, inserimento e cancellazione. L'iterazione costa $\mathcal{O}(n)$ e mantiene l'ordinamento naturale delle chiavi.

Implementazioni: `TreeSet` in Java, `OrderedSet` in Python, `std::set` in C++ STL.

- **Tabelle Hash:** Offrono costo atteso $\mathcal{O}(1)$ per ricerca, inserimento e cancellazione. L'iterazione costa $\mathcal{O}(m)$ e l'ordinamento non è preservato.

Implementazioni: `HashSet` in Java, `set` in Python, `std::unordered_set` in C++ STL.

26.5 Riepilogo Asintotico delle Implementazioni

La seguente tabella riassume i costi computazionali delle diverse implementazioni per Insiemi/Dizionari, dove m rappresenta la dimensione del vettore booleano o della tabella hash e n il numero di elementi inseriti.

Struttura	contains	insert	remove	min	foreach	Ordine
Vettore booleano	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(m)$	$\mathcal{O}(m)$	Sì
Lista non ordinata	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	No
Lista ordinata	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Sì
Vettore ordinato	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Sì
Alberi bilanciati	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	Sì
Hash (Mem. interna)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(m)$	$\mathcal{O}(m)$	No
Hash (Mem. esterna)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(m + n)$	$\mathcal{O}(m + n)$	No

26.6 Costi Computazionali in Python

Di seguito vengono analizzati i costi per le strutture dati native `list` (vettore dinamico) e `set` (basato su hash table) nel linguaggio Python.

Complessità di Python `list` ($n = \text{len}(L)$, $k = \text{lunghezza operando}$):

- **Caso ammortizzato $\mathcal{O}(1)$:** `L.append(x)`, `L[i]` (Index), `len(L)`.
- **Caso $\mathcal{O}(k)$:** `L[i:i+k]` (Slicing), `L.extend(s)`.
- **Caso pessimo $\mathcal{O}(n)$:** `L.copy()`, `L.insert(i, x)`, `L.remove(x)`, `for x in L` (Iterator), `x in L` (Contains), `min(L)` / `max(L)`.

Complessità di Python `set` ($n = \text{len}(S)$, $m = \text{len}(T)$):

Codice	Operazione	Caso medio	Caso pessimo
<code>x in S</code>	Contains	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>for x in S</code>	Iterator	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>S.add(x)</code>	Insert	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>S.remove(x)</code>	Remove	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>S T</code>	Union	$\mathcal{O}(n + m)$	$\mathcal{O}(n \cdot m)$
<code>S & T</code>	Intersection	$\mathcal{O}(\min(n, m))$	$\mathcal{O}(n \cdot m)$
<code>S - T</code>	Difference	$\mathcal{O}(n)$	$\mathcal{O}(n \cdot m)$

27 Grafi

Un grafo è una struttura dati non lineare fondamentale per modellare relazioni tra entità. Le applicazioni sono vastissime: reti sociali (es. gradi di separazione su LinkedIn), dipendenze software (Makefiles, linker), model checking e reti stradali.

27.1 Definizioni Formali

Un grafo è definito come una coppia $G = (V, E)$, dove:

- V è un insieme di **nodi** (o vertici).
- E è un insieme di coppie di nodi, dette **archi** (o lati).

Classificazione principale:

- **Grafi Orientati (Directed):** Le coppie di E sono *ordinate*. L'arco (u, v) è distinto da (v, u) ed è detto incidente *da* u *a* v .
- **Grafi Non Orientati (Undirected):** Le coppie di E *non* sono ordinate. L'arco (u, v) è equivalente a (v, u) e la relazione di adiacenza è simmetrica.

Metriche del grafo:

- **Adiacenza:** Un nodo v è adiacente a u se esiste l'arco (u, v) .
- **Grado (Degree):** Numero di archi incidenti su un nodo (nei grafi orientati si divide in *in-degree* e *out-degree*).
- $n = |V|$ è il numero di nodi.
- $m = |E|$ è il numero di archi. In un grafo denso $m = \mathcal{O}(n^2)$, mentre in un grafo sparso $m = \mathcal{O}(n)$ o $\mathcal{O}(n \log n)$.

28 Memorizzazione e Implementazione

Esistono due approcci principali per memorizzare un grafo nella RAM, con caratteristiche asintotiche opposte.

28.1 Matrice di Adiacenza

Si utilizza una matrice quadrata $n \times n$. Se G è pesato con funzione di peso w , la cella contiene il peso, altrimenti contiene un booleano 1/0:

$$m_{uv} = \begin{cases} w(u, v) & \text{se } (u, v) \in E \\ 0 \text{ (o } +\infty) & \text{se } (u, v) \notin E \end{cases}$$

- **Spazio occupato:** $\mathcal{O}(n^2)$.
- **Verifica adiacenza:** $\mathcal{O}(1)$.
- **Iterazione su tutti gli archi:** $\mathcal{O}(n^2)$.
- *Ideale per:* Grafi densi.

28.2 Liste di Adiacenza (o Vettori di Adiacenza)

Ogni nodo u possiede una lista contenente tutti e soli i nodi v per i quali esiste un arco (u, v) . Se il grafo è pesato, si memorizza la coppia (v, w) .

- **Spazio occupato:** $\mathcal{O}(n + m)$.
- **Verifica adiacenza:** $\mathcal{O}(n)$ nel caso pessimo (bisogna scorrere la lista).
- **Iterazione su tutti gli archi:** $\mathcal{O}(n + m)$.
- *Ideale per:* Grafi sparsi e implementazioni algoritmiche. (Spesso in Python si utilizza un dizionario di dizionari per unire l'efficienza $\mathcal{O}(1)$ del lookup alla flessibilità dello spazio).

29 Visite dei Grafi (Graph Traversal)

29.1 Visita in Ampiezza (Breadth-First Search - BFS)

Esplora il grafo procedendo per "livelli" (o cerchi concentrici) a partire da una sorgente r . Utilizza una **Coda** (**Queue**).

- **Applicazioni:** Calcolare il cammino più breve (in numero di archi) tra r e gli altri nodi (es. calcolo del *Numero di Erdős*). Genera un albero di copertura (BFS Tree).
- **Complessità:** $\mathcal{O}(n + m)$, poiché ogni nodo viene accodato una sola volta e i suoi archi in uscita ispezionati una sola volta.

29.2 Visita in Profondità (Depth-First Search - DFS)

Esplora il grafo seguendo ogni ramo fino in fondo prima di tornare indietro. Si basa su uno **Stack** (implicito tramite chiamate ricorsive, o esplicito iterativo per evitare *StackOverflow* su grafi enormi). Durante la DFS è utile classificare gli archi incontrati (archi dell'albero DFS, archi in avanti, archi all'indietro, archi di attraversamento) in base al "tempo di scoperta" (**discovery time**) e "tempo di fine" (**finish time**) dei nodi.

- **Complessità:** $\mathcal{O}(n + m)$.

30 Applicazioni Pratiche della DFS

30.1 1. Componenti Connesse (Grafì Non Orientati)

Un grafo non orientato è *connesso* se ogni suo nodo è raggiungibile da ogni altro nodo. Una *componente connessa* è un sottografo connesso e massimale.

- L'algoritmo esegue una DFS a partire da un nodo non marcato, assegnando un ID univoco a tutti i nodi scoperti in quella sessione. Se restano nodi non marcati, incrementa l'ID e riavvia la DFS per mappare le restanti componenti.

30.2 2. Ciclicità e Grafi Aciclici (DAG)

Un **ciclo** è un cammino in cui il primo e l'ultimo nodo coincidono.

- **Grafo non orientato:** Si ha un ciclo se durante la DFS si incontra un nodo già visitato (che non sia il padre diretto da cui si proviene).
- **Grafo orientato aciclico (DAG):** Un grafo orientato ha un ciclo *se e solo se* la DFS rileva un **arco all'indietro** (cioè un arco che punta verso un "antenato" nell'albero di ricerca, individuabile se l'antenato non ha ancora concluso il suo **finish time**).

30.3 3. Ordinamento Topologico (solo per DAG)

Un ordinamento topologico è una disposizione lineare dei nodi tale che se esiste un arco (u, v) , allora u appare *prima* di v nell'ordinamento. Non può esistere in presenza di cicli.

- **Algoritmo DFS-based:** Esegue una DFS. Nel momento della "post-visita" (tempo di fine, quando tutti i discendenti sono stati visitati), inserisce il nodo in cima a uno Stack (ordinamento LIFO).
- **Applicazioni:** Risoluzione dipendenze in pacchetti software, Makefile, calcolo celle spreadsheet.

30.4 4. Componenti Fortemente Connesse (SCC - Grafi Orientati)

Un grafo orientato è fortemente connesso se ogni nodo è mutuamente raggiungibile da ogni altro nodo. Una **SCC** è un sottografo fortemente connesso e massimale. Identificarle richiede una procedura specifica. La soluzione più elegante è l'**Algoritmo di Kosaraju (1978)**:

1. Eseguire un Ordinamento Topologico sul grafo G (tramite DFS).
2. Calcolare il **Grafo Trasposto** G^T invertendo la direzione di tutti gli archi ($\mathcal{O}(n+m)$).
3. Eseguire l'algoritmo delle componenti connesse standard sul grafo G^T , scegliendo però le radici di partenza in base all'ordine LIFO restituito dallo Stack generato al punto 1.

La complessità totale rimane rigorosamente $\mathcal{O}(n+m)$. (Esiste anche l'Algoritmo di Tarjan, anch'esso $\mathcal{O}(n+m)$, che evita la creazione del grafo trasposto eseguendo una singola passata).

31 Strutture Dati Speciali: Code con Priorità

In molte applicazioni non è necessario avere a disposizione l'intero set di operazioni di un dizionario (come la ricerca di elementi arbitrari); in questi casi, è possibile progettare strutture dati *specializzate* e molto più efficienti.

Definizione: Coda con Priorità (Priority Queue)

Una coda con priorità è una struttura dati astratta simile ad una coda (Queue), in cui però ogni elemento inserito possiede una **priorità** associata. L'estrazione non avviene in base all'ordine di arrivo (FIFO), ma in base alla priorità:

- **Min-priority queue:** Estrazione per valori *crescenti* di priorità (il minimo viene estratto per primo).
- **Max-priority queue:** Estrazione per valori *decrescenti* di priorità (il massimo viene estratto per primo).

Le applicazioni tipiche includono simulatori *event-driven* (dove la priorità è il timestamp dell'evento), l'algoritmo di Dijkstra per i cammini minimi, la codifica di Huffman e l'algoritmo di Prim.

31.1 Specifica del Tipo di Dato

```
% Pseudocodice: Specifica di una Min-Priority Queue
PRIORITYQUEUE PriorityQueue(int n)      % Crea coda vuota di capacità n
boolean isEmpty()                       % Verifica se vuota
item min()                              % Legge l'elemento minimo
item deleteMin()                        % Rimuove e restituisce il minimo
PRIORITYITEM insert(item x, int p)      % Inserisce x con priorità p
decrease(PRIORITYITEM y, int p)         % Abbassa la priorità dell'oggetto y a p
```

31.2 L'Albero Heap

Confrontando le possibili implementazioni, si nota che vettori ordinati e liste non offrono un bilanciamento ottimale tra i costi di inserimento ed estrazione. L'**Heap** è una struttura inventata da J. Williams (1964) che unisce i vantaggi di un albero (esecuzione in $\mathcal{O}(\log n)$) ai vantaggi di un vettore (memorizzazione contigua ed efficiente).

Per definire un Heap, dobbiamo ricordare due tipologie di alberi:

1. **Albero binario perfetto:** Tutte le foglie hanno la stessa profondità h e i nodi interni hanno grado 2. Ha $n = 2^{h+1} - 1$ nodi.
2. **Albero binario completo:** Tutte le foglie hanno profondità h o $h - 1$. Tutti i nodi al livello h sono "accatastati" a sinistra.

Proprietà Heap

Un albero **max-heap** (o **min-heap**) è un **albero binario completo** tale che il valore memorizzato in ogni nodo è **maggiore** (o **minore**) dei valori memorizzati nei suoi figli.

Questa proprietà non impone un ordinamento totale tra i rami (è un *ordinamento parziale*).

31.2.1 Memorizzazione su Vettore (Vettore Heap)

Grazie alla struttura completa dell'albero, un heap viene memorizzato compattamente in un **vettore** A , usando indici matematici per muoversi tra padre e figli:

- Radice: $root() = 0$
- Padre del nodo i : $p(i) = \lfloor (i - 1)/2 \rfloor$
- Figlio sinistro del nodo i : $l(i) = 2i + 1$
- Figlio destro del nodo i : $r(i) = 2i + 2$

31.3 L'algoritmo HeapSort

HeapSort ordina un vettore *in-place* costruendo prima un max-heap e poi spostando iterativamente l'elemento massimo in ultima posizione.

31.3.1 1. Ripristino della Proprietà Heap (**maxHeapRestore**)

Se un nodo i viola la proprietà heap rispetto ai suoi figli (ma i suoi sottoalberi sono heap validi), si scambia il nodo con il maggiore dei suoi figli e si propaga la correzione verso il basso.

```
% Ripristina la proprietà max-heap scendendo nell'albero
maxHeapRestore(item[] A, int i, int dim)
int max = i
if l(i) < dim and A[l(i)] > A[max] then max = l(i)
if r(i) < dim and A[r(i)] > A[max] then max = r(i)
if i != max then
    swap(A, i, max)
    maxHeapRestore(A, max, dim) % Chiamata ricorsiva sul figlio
```

Costo computazionale: Poiché l'albero ha altezza $\lfloor \log n \rfloor$, l'algoritmo costa $\mathcal{O}(\log n)$.

31.3.2 2. Costruzione dell'Heap (**heapBuild**)

Costruisce un max-heap a partire da un vettore non ordinato.

```
% Costruisce un max-heap procedendo dal basso verso l'alto
heapBuild(item[] A, int n)
% I nodi da n/2 a n-1 sono foglie (heap di 1 elemento validi)
for i = floor(n/2) - 1 downto 0 do
    maxHeapRestore(A, i, n)
```

Costo computazionale: Sebbene sembri $\mathcal{O}(n \log n)$, un'analisi matematica rigorosa dimostra che **maxHeapRestore** viene eseguito su alberi via via più grandi, ma la maggior parte dei nodi è vicina alle foglie. La serie converge e il costo totale è strettamente $\Theta(n)$.

31.3.3 3. Algoritmo di Ordinamento Finale (heapsort)

```
heapsort(item[] A, int n)
heapBuild(A, n)
for capacity = n - 1 downto 1 do
    swap(A, 0, capacity)          % Sposta il massimo in fondo
    maxHeapRestore(A, 0, capacity) % Ripristina l'heap sulla parte restante
```

Costo globale: $\mathcal{O}(n)$ per la costruzione + $(n - 1) \times \mathcal{O}(\log n)$ per i ripristini = $\Theta(n \log n)$.
Gode di $\mathcal{O}(1)$ memoria ausiliaria ed è usato nel kernel Linux.

31.4 Implementazione della Coda con Priorità (Min-Heap)

Per realizzare la coda con priorità, i nodi memorizzano non solo il valore e la priorità, ma anche la loro **posizione corrente** nel vettore. Questo è indispensabile per la funzione decrease.

```
PRIORITYITEM
int priority
item value
int pos      % Posizione attuale nel vettore heap

% Inserimento in fondo e risalita (up-heap)
PRIORITYITEM insert(item x, int p)
% ... inizializza H[dim] con x, p, e pos = dim
int i = dim
dim = dim + 1
% Se è minore del padre, sale verso l'alto
while i > 0 and H[i].priority < H[p(i)].priority do
    swap(H, i, p(i))
    i = p(i)
return H[i]

% Estrazione del minimo e discesa (down-heap tramite Restore)
item deleteMin()
item temp = H[0].value
dim = dim - 1
swap(H, 0, dim)
minHeapRestore(H, 0, dim)
return temp
```

Complessità Operazioni Priority Queue:

- min(): $\Theta(1)$
- insert(), deleteMin(), decrease(): $\mathcal{O}(\log n)$

32 Insiemi Disgiunti (Merge-Find Set)

Definizione

Un *Merge-Find Set* serve a gestire una collezione $S = \{S_1, S_2, \dots, S_k\}$ di insiemi **dinamici e mutuamente disgiunti** ($\forall i \neq j \implies S_i \cap S_j = \emptyset$). Ogni insieme è identificato da un **rappresentante univoco** scelto tra i suoi membri.

Operazioni supportate:

- `Mfset(int n)`: Crea n insiemi disgiunti composti da un solo elemento $\{0\}, \dots, \{n-1\}$.
- `find(int x)`: Restituisce il rappresentante dell'insieme a cui appartiene x .
- `merge(int x, int y)`: Unisce l'insieme contenente x con l'insieme contenente y .

Applicazione principe: Calcolo dinamico delle componenti connesse in un grafo non orientato (utile nell'algoritmo di Kruskal).

32.1 1. Realizzazione basata su Liste

Ogni insieme è una **lista concatenata**. La testa della lista funge da rappresentante. Ogni nodo ha un puntatore al successivo e un **puntatore diretto al rappresentante**.

- `find(x)`: Costa $\mathcal{O}(1)$ seguendo il puntatore diretto alla testa.
- `merge(x, y)`: Appende la lista di y in coda a quella di x . Richiede di aggiornare tutti i puntatori al rappresentante nella lista "appesa". Costo pessimo $\mathcal{O}(n^2)$ per n operazioni, costo ammortizzato $\mathcal{O}(n)$.

32.2 2. Realizzazione basata su Alberi (Foresta)

Ogni insieme è un albero. La **radice è il rappresentante**. I nodi possiedono **solo un puntatore verso il padre**.

- `merge(x, y)`: Modifica il puntatore del padre della radice di y , agganciandolo alla radice di x . Costa $\mathcal{O}(1)$.
- `find(x)`: Risale la catena dei padri fino alla radice. Costa $\mathcal{O}(n)$ nel caso pessimo (albero degenerare a lista).

32.3 Ottimizzazioni: Euristiche

Per migliorare le performance, si utilizzano algoritmi euristici.

32.3.1 Euristica sul Peso (per Liste)

Per ottimizzare la *merge*, le liste memorizzano la loro lunghezza. Si **aggancia sempre la lista più corta a quella più lunga**.

- In questo modo, un nodo vede cambiare il proprio rappresentante solo quando la dimensione del suo insieme almeno raddoppia.
- **Vantaggio:** Il costo di *merge* scende a $\mathcal{O}(\log n)$ ammortizzato.

32.3.2 Euristicica sul Rango (per Alberi)

Ogni nodo mantiene il proprio **rango** (limite superiore all'altezza del sottoalbero). Si aggancia l'albero con rango *più basso* alla radice di quello con rango *più alto*. Se i ranghi sono uguali, se ne aggancia uno all'altro e il rango del vincitore aumenta di 1.

- Si dimostra che un albero MFSET con radice r ha almeno $2^{\text{rank}[r]}$ nodi.
- **Vantaggio:** L'altezza massima dell'albero è limitata a $\log n$. Il costo di `find` scende a $\mathcal{O}(\log n)$.

32.3.3 Euristicica della Compressione dei Cammini (per Alberi)

Durante l'operazione `find`, l'albero viene "appiattito": tutti i nodi visitati durante la risalita vengono **agganciati direttamente alla radice**.

```
% Find ricorsiva con compressione dei cammini
int find(int x)
if p[x] != x then
p[x] = find(p[x]) % Aggiorna il padre collegandolo alla radice
return p[x]
```

32.3.4 Combinazione: Rango + Compressione

Applicando entrambe le euristiche agli alberi, si ottiene la struttura definitiva per i Merge-Find Set.

- **Complessità Ammortizzata:** Eseguire m operazioni `merge/find` su n elementi ha un costo complessivo $\mathcal{O}(m \cdot \alpha(n))$.
- $\alpha(n)$ è la **funzione inversa di Ackermann**, che cresce così lentamente da risultare ≤ 5 per qualsiasi valore di $n \leq 2^{65536}$. Pertanto, il costo delle operazioni può essere considerato $\mathcal{O}(1)$ a fini pratici.

33 Il Problema dei Cammini Minimi e la Scelta della Struttura Dati

L'obiettivo di questa sezione è dimostrare come la scelta della struttura dati sia una tecnica algoritmica fondamentale che può alterare drasticamente la complessità di un algoritmo. Questo concetto viene illustrato attraverso il **problema dei cammini minimi (Shortest Path)**.

Definizione: Cammini Minimi da Singola Sorgente (SSSP)

Input:

- Un grafo orientato $G = (V, E)$.
- Un nodo sorgente s .
- Una funzione di peso $w : E \rightarrow \mathbb{R}$.

Definizione di Costo: Dato un cammino $p = \langle v_0, v_1, \dots, v_k \rangle$, il suo costo è la somma dei pesi degli archi: $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$. **Output:** Trovare, per ogni nodo $u \in V$, un cammino da s ad u il cui costo sia **minimo** tra tutti i possibili cammini.

33.1 Caratteristiche delle Soluzioni

- **Pesi:** Gli algoritmi differiscono a seconda che i pesi siano solo positivi o anche negativi. La presenza di **cicli negativi** rende il problema mal definito, in quanto si potrebbe percorrere il ciclo all'infinito per abbassare il costo.
- **Sottostruttura ottima:** Un sotto-cammino di un cammino minimo è necessariamente esso stesso un cammino minimo.
- **Albero dei cammini minimi:** Grazie alla sottostruttura ottima, l'insieme dei cammini minimi da s a tutti i nodi raggiungibili forma un albero di copertura radicato in s . La soluzione è quindi descritta da un vettore delle distanze d e da un vettore dei padri T .

34 Schema Generico e Teorema di Bellman

34.1 Il Teorema di Bellman

Come riconoscere se una soluzione ammissibile (descritta da un albero T e da un vettore d) è effettivamente quella ottima? Il **Teorema di Bellman** afferma che una soluzione ammissibile T è ottima **se e solo se**:

- **Coerenza:** $d[v] = d[u] + w(u, v) \quad \forall (u, v) \in T$.
- **Nessun miglioramento possibile:** $d[v] \leq d[u] + w(u, v) \quad \forall (u, v) \in E$.

Se la seconda condizione è violata, significa che passare per u offre un cammino più breve per raggiungere v .

34.2 Algoritmo Prototipo e Rilassamento

Partendo dal Teorema di Bellman, si definisce l'operazione di **rilassamento**. Si inizializza la distanza di s a 0 e di tutti gli altri nodi a $+\infty$. Finché esiste un arco (u, v) che viola la disuguaglianza di Bellman, se ne "rilassa" la tensione aggiornando la distanza.

```
% Algoritmo generico per i Cammini Minimi
(int[], int[]) shortestPath(GRAPH G, NODE s)
% Inizializzazione...
DATASTRUCTURE S = DataStructure(); S.add(s)

while not S.isEmpty() do
    int u = S.extract()
    b[u] = false
    foreach v in G.adj(u) do
        % Condizione di Rilassamento di Bellman
        if d[u] + G.w(u,v) < d[v] then
            if not b[v] then
                S.add(v)
                b[v] = true
            else
                % Azione se v è già in S
                T[v] = u
                d[v] = d[u] + G.w(u,v)
    return (T, d)
```

Cambiando la struttura dati DATASTRUCTURE S , si ottengono algoritmi profondamente diversi.

35 Algoritmi per Sorgente Singola (SSSP)

35.1 Algoritmo di Dijkstra (1959) - Pesì Positivi

Sostituendo **S** con una **Coda con Priorità (Priority Queue)**, otteniamo l'algoritmo di Dijkstra. Funziona correttamente **solo con pesi non negativi**. È basato su una strategia greedy: ogni volta che un nodo u viene estratto, la sua distanza $d[u]$ è garantita essere quella minima definitiva.

La complessità dipende dall'implementazione della coda con priorità:

- **Vettore non ordinato (Dijkstra originale):** Estrazione costa $\mathcal{O}(n)$, diminuzione priorità costa $\mathcal{O}(1) \implies \mathcal{O}(n^2)$. Ideale per grafi densi.
- **Heap Binario (Algoritmo di Johnson, 1977):** Estrazione e decremento priorità costano $\mathcal{O}(\log n) \implies \mathcal{O}(m \log n)$. Ottimale per grafi sparsi.
- **Heap di Fibonacci (Fredman-Tarjan, 1987):** Decremento costa $\mathcal{O}(1)$ ammortizzato $\implies \mathcal{O}(m + n \log n)$. Ideale per grafi enormi e densi.

35.2 Algoritmo di Bellman-Ford-Moore (1958) - Pesì Negativi

Sostituendo **S** con una **Coda semplice (Queue - FIFO)**, si ottiene la variante SPFA (Shortest Path Faster Algorithm) di Bellman-Ford.

- **Funzionamento:** Un nodo può essere inserito ed estratto dalla coda fino a $n - 1$ volte. Al termine della k -esima passata sulla coda, le distanze calcolate sono i cammini minimi formati da al più k archi.
- **Vantaggio:** Gestisce correttamente i **pesi negativi**.
- **Svantaggio:** Costo computazionale pesante pari a $\mathcal{O}(n \cdot m)$.

35.3 Caso Speciale: Grafi Orientati Aciclici (DAG)

In un DAG non esistono cicli (e quindi neanche cicli negativi), quindi il problema è più semplice.

- **Algoritmo:** Sostituendo **S** con lo **Stack dell'ordinamento topologico**, si estraggono e rilassano gli archi una volta sola seguendo la direzionalità imposta dal grafo.
- **Complessità:** Estremamente rapido, tempo lineare $\mathcal{O}(m + n)$.

36 Cammini Minimi per Tutte le Coppie (APSP)

Se si vuole calcolare la distanza minima tra **tutte** le coppie di vertici, si può applicare ripetutamente un algoritmo a singola sorgente per ogni nodo ($\mathcal{O}(n \cdot m \log n)$ per grafi sparsi positivi), ma in presenza di pesi negativi su grafi densi è più efficiente ricorrere alla **Programmazione Dinamica**.

36.1 Algoritmo di Floyd-Warshall (1962)

Floyd-Warshall lavora sul concetto di **cammino minimo k -vincolato**. Un cammino p_{xy}^k è il percorso minimo tra x e y che passa *esclusivamente* per nodi intermedi appartenenti al set ristretto $\{v_0, \dots, v_{k-1}\}$.

Formulazione Ricorsiva:

$$d^k[x][y] = \begin{cases} w(x, y) & \text{se } k = 0 \\ \min(d^{k-1}[x][y], d^{k-1}[x][k-1] + d^{k-1}[k-1][y]) & \text{se } k > 0 \end{cases}$$

- Il caso base ($k = 0$) rappresenta i collegamenti diretti tra due nodi senza intermediari.
- Il passo ricorsivo controlla se includere il vertice ($k - 1$) come tappa intermedia migliora la distanza precedente.

```
% Programmazione Dinamica -  $\mathcal{O}(n^3)$ 
(int[] [], int[] []) floydWarshall(GRAPH G)
% d: matrice delle distanze, T: matrice dei padri
% Inizializzazione matrice d con pesi diretti w(u,v) e T con i padri diretti

for k = 1 to G.n do
    foreach u in G.V() do
        foreach v in G.V() do
            if d[u][k-1] + d[k-1][v] < d[u][v] then
                d[u][v] = d[u][k-1] + d[k-1][v]
                T[u][v] = T[k-1][v]

return (d, T)
```

Complessità: Triplo ciclo annidato sui vertici $\implies \mathcal{O}(n^3)$. Estremamente elegante ed efficiente per grafi densi con pesi negativi.

36.2 Chiusura Transitiva (Algoritmo di Warshall)

Derivato dalla logica precedente, calcola il grafo orientato $G^* = (V, E^*)$ in cui l'arco $(u, v) \in E^*$ se e solo se esiste un qualsiasi cammino (di lunghezza ≥ 1) da u a v in G . Si implementa con una logica booleana (AND/OR logici) anziché somme aritmetiche, mantenendo un costo di $\mathcal{O}(n^3)$:

$$M^k[x][y] = \begin{cases} y \in G.adj(x) & k = 0 \\ M^{k-1}[x][y] \vee (M^{k-1}[x][k-1] \wedge M^{k-1}[k-1][y]) & k > 0 \end{cases}$$

37 Metodologia di Risoluzione dei Problemi

Di fronte a un problema computazionale, non esistono "ricette generali" per risolverlo in modo efficiente, ma il processo di progettazione si divide in quattro fasi principali:

1. **Classificazione del problema.**
2. **Caratterizzazione della soluzione.**
3. **Scelta della tecnica di progetto.**
4. **Utilizzo di strutture dati adeguate.**

37.1 Classificazione dei Problemi

I problemi si dividono in tre grandi categorie:

- **Problemi decisionali:** Verificano se il dato di ingresso soddisfa una certa proprietà. La soluzione è strettamente *Si/No* (es. Stabilire se un grafo è connesso).
- **Problemi di ricerca:** Dato uno spazio di ricerca, mirano a trovare una "soluzione ammissibile" che rispetti determinati vincoli (es. trovare la posizione di una sottostringa in un testo).
- **Problemi di ottimizzazione:** Ogni soluzione ammissibile è associata a una **funzione di costo**. L'obiettivo è trovare la soluzione con il costo minimo o massimo assoluto (es. il cammino più breve tra due nodi).

La rigorosa definizione matematica del problema può suggerire direttamente la tecnica di risoluzione ottima (es. la presenza di una *sottostruttura ottima* suggerisce l'uso della programmazione dinamica, mentre una *proprietà greedy* porta all'omonima tecnica).

38 La Tecnica del Divide-et-Impera

Il **Divide-et-impera** è una tecnica top-down applicata tipicamente a problemi di decisione e ricerca. Si struttura rigorosamente in tre fasi:

1. **Divide:** Si divide il problema di partenza in sotto-problemi più piccoli e reciprocamente indipendenti.
2. **Impera:** Si risolvono i sotto-problemi in modo ricorsivo.
3. **Combina:** Si uniscono le soluzioni dei sotto-problemi per ottenere la soluzione del problema originale.

Quando applicarlo: Questa tecnica risulta vantaggiosa solo quando i passi di *divide* e *combina* sono operativamente semplici e il costo asintotico globale è strettamente inferiore a quello del corrispondente algoritmo iterativo naif. Ad esempio, cercare il minimo in un vettore tramite divide-et-impera produce la ricorrenza $T(n) = 2T(n/2) + 1 = \Theta(n)$, non portando alcun vantaggio rispetto alla banale scansione iterativa. Al contrario, porta enormi vantaggi nell'ordinamento (es. Merge Sort, Quicksort). Inoltre, il divide-et-impera facilita la **parallelizzazione** e favorisce un utilizzo ottimale della cache di memoria (*cache oblivious*).

38.1 Esempio Introduttivo: Torri di Hanoi

Un classico problema risolvibile unicamente con questa tecnica è il gioco delle Torri di Hanoi: spostare n dischi da un piolo sorgente a una destinazione usando un piolo di appoggio, muovendo un disco alla volta senza mai mettere un disco più grande sopra uno più piccolo.

```
% Pseudocodice Torri di Hanoi
hanoi(int n, int src, int dest, int middle)
if n == 1 then
  print src -> dest
else
  hanoi(n - 1, src, middle, dest) % n-1 dischi da src a middle
  print src -> dest                % il disco più grande da src a dest
  hanoi(n - 1, middle, dest, src) % n-1 dischi da middle a dest
```

Complessità: L'equazione di ricorrenza generata è $T(n) = 2T(n - 1) + 1$, la cui forma chiusa è $\mathcal{O}(2^n)$. Tale soluzione esponenziale è matematicamente l'ottimo per questo specifico problema.

39 Quicksort (Hoare, 1961)

Il Quicksort è un celebre algoritmo di ordinamento *in-memory* (non richiede vettori di appoggio ausiliari) basato sul divide-et-impera. Le sue fasi sono sbilanciate rispetto al Merge Sort:

- **Divide (Fase complessa):** Sceglie un valore detto **perno (pivot)** all'interno del vettore e sposta gli elementi in modo da isolare il pivot nella sua posizione definitiva j . A sinistra del pivot ci saranno solo valori $< pivot$, a destra solo valori $\geq pivot$.
- **Impera:** Richiama ricorsivamente il Quicksort sui due sottovettori (escluso il pivot).
- **Combina (Fase nulla):** Non fa nulla, poiché ordinare i due sottovettori partizionati implica implicitamente ordinare l'intero vettore.

39.1 Implementazione

```
% Procedura principale
QuickSort(item[] A, int start, int end)
if start < end then
    int j = perno(A, start, end)
    QuickSort(A, start, j - 1)
    QuickSort(A, j + 1, end)

% Funzione di partizionamento: sposta gli elementi e trova l'indice j
int perno(item[] A, int start, int end)
item pivot = A[start] % Scelta naif: il primo elemento è il pivot
int j = start
for i = start + 1 to end do
    if A[i] < pivot then
        j = j + 1
        swap(A, i, j)
% Posiziona il pivot esattamente in mezzo ai due insiemi creati
swap(A, start, j)
return j
```

39.2 Analisi della Complessità

Il costo della funzione `perno()` è strettamente $\Theta(n)$ per la scansione lineare dell'intervallo.

- **Caso Ottimo:** Ad ogni passo il pivot divide esattamente il vettore a metà. $T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log n)$.
- **Caso Pessimo:** Il vettore è già ordinato (o ordinato al contrario) e la divisione produce sottovettori di dimensione 0 e $n - 1$. $T(n) = T(n - 1) + T(0) + \Theta(n) \implies \Theta(n^2)$.
- **Caso Medio:** Il comportamento reale è molto più vicino al caso ottimo. Anche partizionamenti fortemente sbilanciati (es. 99-a-1) producono una complessità $\Theta(n \log n)$ poiché limitano la profondità dell'albero delle chiamate in modo logaritmico. Statisticamente, i partizionamenti mediamente bilanciati dominano su quelli pessimi, mantenendo l'attesa a $\Theta(n \log n)$.

Miglioramento Euristico (Selezione del Pivot): Per evitare il caso pessimo su array quasi ordinati, anziché prendere il primo elemento $A[\text{start}]$, si seleziona il **valore mediano** tra il primo elemento, l'ultimo e quello centrale, posizionandolo poi in testa al vettore prima di lanciare la scansione.

40 Algoritmo di Strassen (Moltiplicazione di Matrici)

Il prodotto classico fra due matrici A e B quadrate $n \times n$ costa, calcolando riga per colonna, $\Theta(n^3)$. Applicando un approccio divide-et-impera elementare, si suddividono le matrici $n \times n$ in 4 sottomatrici $\frac{n}{2} \times \frac{n}{2}$ e si eseguono 8 moltiplicazioni. L'equazione di ricorrenza $T(n) = 8T(n/2) + n^2$ porta comunque a una complessità $\Theta(n^3)$.

L'intuizione di Strassen (1969): Strassen ha dimostrato che manipolando algebricamente i blocchi, è possibile ricalcolare la matrice finale C effettuando **solo 7 moltiplicazioni ricorsive** al posto di 8, a costo di un maggior numero di addizioni/sottrazioni (costo $\Theta(n^2)$).

$$T(n) = 7T(n/2) + \Theta(n^2)$$

Risolvendo la ricorrenza con il Master Theorem:

$$\alpha = \log_2 7 \approx 2.81 \implies T(n) = \Theta(n^{2.81})$$

Nota pratica: Pur essendo asintoticamente superiore, l'algoritmo di Strassen ha costanti moltiplicative alte e una stabilità numerica inferiore. La *rule-of-thumb* è utilizzarlo solo per matrici di grosse dimensioni ($n \geq 100$). Al giorno d'oggi, la ricerca matematica ha abbassato ulteriormente questo limite (es. algoritmi della famiglia Coppersmith e Winograd fino a $\approx \mathcal{O}(n^{2.37})$), ma l'algoritmo standard e quello di Strassen restano i più implementati.

41 Applicazione Pratica: Ricerca di un Gap

Esercizio: Trovare il Gap

Definizione: In un vettore V contenente $n \geq 2$ interi, un **gap** è un indice i ($0 < i \leq n - 1$) tale che $V[i - 1] < V[i]$. **Teorema:** Se $n \geq 2$ e $V[0] < V[n - 1]$, allora V contiene con certezza almeno un gap.

Dimostrazione per assurdo: Se non ci fossero gap, ogni elemento sarebbe decrescente o uguale al successivo: $V[0] \geq V[1] \geq \dots \geq V[n - 1]$. Ciò contraddice palesemente la premessa $V[0] < V[n - 1]$.

Soluzione Algoritmica (Divide-et-Impera): Sfruttando questa proprietà topologica, possiamo trattare il vettore come uno spazio dicotomico, portando la ricerca da $\mathcal{O}(n)$ (scansione lineare) a $\mathcal{O}(\log n)$.

```
% Algoritmo efficiente  $\mathcal{O}(\log n)$ 
int gap(int[] V, int n)
return gapRec(V, 1, n - 1) % Nota: l'intervallo è tra gli indici 1 e n-1

int gapRec(int[] V, int i, int j)
% Caso base: sottovettore di due soli elementi
if j == i + 1 then
return j

% Caso ricorsivo
int m = floor((i + j) / 2)

% Se l'estremo destro è maggiore del punto medio,
% per il teorema esiste di sicuro un gap a destra
if V[m] < V[j] then
    return gapRec(V, m, j)
% Altrimenti, se  $V[m] \geq V[j]$ , sapendo che originariamente  $V[i] < V[j]$ ,
% per transitività deve esserci una "risalita" a sinistra.
else
    return gapRec(V, i, m)
```

42 Programmazione Dinamica

Principi della Programmazione Dinamica

La Programmazione Dinamica è una tecnica di risoluzione dei problemi, prevalentemente usata per l'**ottimizzazione**. Si basa sul principio per cui un problema complesso viene spezzato in **sottoproblemi ripetuti**. Per evitare di ricalcolare più volte lo stesso sottoproblema, **ogni sottoproblema viene risolto una volta sola e la sua soluzione viene memorizzata in una tabella (array o matrice)**. L'iterazione avviene in logica **bottom-up**: si parte dai problemi più piccoli (casi base) e si sale calcolando i problemi via via più grandi.

"Those who cannot remember the past are condemned to repeat it" – George Santayana

La tecnica **top-down** parallela alla programmazione dinamica prende il nome di **Memoization (Annotazione)**: la formula resta ricorsiva, ma prima di calcolarla si verifica se il risultato è già salvato in tabella.

42.1 Approccio Metodologico

Per sviluppare un algoritmo con programmazione dinamica si seguono tre fasi principali:

1. **Definire il valore della soluzione in maniera ricorsiva (Sottostruttura ottima)**: Formulare l'equazione di ricorrenza (tipicamente una funzione $DP[\dots]$).
2. **Tabella delle soluzioni**: Costruire un algoritmo iterativo (o ricorsivo memoizzato) per riempire la tabella partendo dai casi base.
3. **Ricostruzione della soluzione**: Retrocedere lungo la tabella DP per identificare quali scelte hanno portato alla soluzione ottima.

43 Esempi Classici

43.1 1. Il Problema del Domino

Problema: Trovare il numero di possibili disposizioni di n tessere 2×1 in un rettangolo di dimensioni $2 \times n$.

Definizione Ricorsiva: Se posiziono l'ultima tessera in verticale, devo riempire lo spazio rimanente di $n - 1$. Se la posiziono in orizzontale, devo obbligatoriamente metterne due orizzontali una sopra l'altra, restando con $n - 2$ da riempire. Le disposizioni si sommano:

$$DP[n] = \begin{cases} 1 & \text{se } n \leq 1 \\ DP[n - 2] + DP[n - 1] & \text{se } n > 1 \end{cases}$$

Questo genera la **Successione di Fibonacci**.

Evoluzione delle Soluzioni:

- **Ricorsione Pura:** Algoritmo naif con costo temporale esponenziale $\mathcal{O}(2^n)$ a causa dei sottoproblemi risolti ripetutamente.
- **Programmazione Dinamica (Vettore completo):** Si alloca un array $DP[0 \dots n]$. Tempo $\mathcal{O}(n)$, Spazio $\mathcal{O}(n)$. (Assumendo costo $\mathcal{O}(1)$ per la somma).
- **Ottimizzazione Spaziale:** Poiché per calcolare il passo i servono solo i passi $i - 1$ e $i - 2$, è sufficiente mantenere in memoria tre variabili intere. Tempo $\mathcal{O}(n)$, Spazio $\mathcal{O}(1)$.

Nota teorica sui calcoli: Se assumiamo un modello di **costo logaritmico** (in cui la somma di due grandi numeri non costa $\mathcal{O}(1)$ ma un tempo proporzionale ai bit, $\Theta(n)$), l'algoritmo dinamico costerebbe $\mathcal{O}(n^2)$.

43.2 2. Il Problema di Hateville

Problema: Una strada con n case. Ogni casa i offre una donazione $D[i]$, ma non si possono accettare donazioni da due case adiacenti. Massimizzare il totale raccolto.

Sottostruttura ottima: Considerando l'iesima casa, si hanno due scelte mutuamente esclusive:

1. **Non la prendo:** Il profitto massimo è pari al profitto ottenuto considerando le prime $i - 1$ case ($DP[i - 1]$).
2. **La prendo:** Prendo $D[i]$ ma non posso prendere la casa $i - 1$. Il profitto totale è il profitto delle prime $i - 2$ case sommato a $D[i]$ ($DP[i - 2] + D[i]$).

$$DP[i] = \begin{cases} 0 & i = 0 \\ D[0] & i = 1 \\ \max(DP[i - 1], DP[i - 2] + D[i - 1]) & i \geq 2 \end{cases}$$

Nota sugli indici: Si usa $D[i - 1]$ per allineare matematicamente l'identificativo 1-based dell'equazione ricorsiva (DP) all'indice 0-based del vettore D .

Ricostruzione della soluzione (Backtracking sulla Tabella): Per risalire alle case effettivamente scelte, si parte dal fondo dell'array $DP[n]$. Se $DP[i] == DP[i - 1]$, significa che la casa i non è stata scelta, e si prosegue ricorsivamente con $i - 1$. Altrimenti, è stata scelta, e si prosegue con $i - 2$. **Complessità:** Tempo $\mathcal{O}(n)$ per la costruzione, Spazio $\mathcal{O}(n)$ (indispensabile per la ricostruzione).

43.3 3. Il Problema dello Zaino (Knapsack 0/1)

Problema: Dato un set di n oggetti, ognuno con un peso $w[i]$ e un profitto $p[i]$, e uno zaino di capacità massima C , massimizzare il profitto senza superare il peso.

Definizione Ricorsiva del Valore: Definiamo $DP[i][c]$ come il massimo profitto ottenibile considerando solo i primi i oggetti, avendo a disposizione una capacità residua c . Per l'oggetto i -esimo:

- Se lo scarto (**Non Preso**): La capacità resta c , e il profitto è lo stesso ottenuto usando $i - 1$ oggetti: $DP[i - 1][c]$.
- Se lo prendo (**Preso**): Consumo la capacità $w[i - 1]$, quindi al valore dell'oggetto $p[i - 1]$ devo sommare il miglior profitto ottenibile con gli $i - 1$ oggetti rimanenti nello spazio residuo $c - w[i - 1]$.

$$DP[i][c] = \begin{cases} 0 & \text{se } i = 0 \text{ o } c = 0 \\ -\infty & \text{se } c < 0 \\ \max(DP[i - 1][c - w[i - 1]] + p[i - 1], DP[i - 1][c]) & \text{altrimenti} \end{cases}$$

```
% Implementazione Bottom-up
int knapsack(int[] w, int[] p, int n, int C)
DP = new int[0...n][0...C]
% Inizializzazione casi base a 0 per i=0 e c=0
for i = 1 to n do
for c = 1 to C do
% Se l'oggetto ci sta nello zaino (capacità residua sufficiente)
if w[i-1] <= c then
DP[i][c] = max(DP[i-1][c-w[i-1]] + p[i-1], DP[i-1][c])
else
% L'oggetto non ci sta, devo scartarlo per forza
DP[i][c] = DP[i-1][c]

return DP[n][C]
```

Complessità: $\mathcal{O}(n \cdot C)$ Questo algoritmo è definito **pseudo-polinomiale**. Non è un algoritmo strettamente polinomiale, perché la complessità dipende dal *valore* di C (che richiede $k = \log_2 C$ bit per essere rappresentato). La complessità reale rispetto alla grandezza dell'input binario è $\mathcal{O}(n \cdot 2^k)$, quindi esponenziale rispetto al numero di bit che rappresentano la capacità dello zaino.

44 Programmazione Dinamica: Memoization e Varianti

44.1 Zaino 0/1 con Memoization

Risolvere il problema dello Zaino in pura ricorsione genera un albero delle chiamate con molti sottoproblemi ripetuti. L'equazione di ricorrenza risultante per il caso puramente ricorsivo è $T(n) = 2T(n-1) + 1$, che porta a una complessità esponenziale $\mathcal{O}(2^n)$. Poiché non tutti gli elementi della matrice programmatica sono necessari per la risoluzione del problema originario, si può ottimizzare l'approccio unendo il top-down (ricorsione) con la memorizzazione tipica della programmazione dinamica: questa tecnica prende il nome di **Memoization** (annotazione).

- Si crea una tabella DP inizializzata con un valore speciale (es. -1) che indica che un sottoproblema non è ancora stato risolto.
- Quando la ricorsione incontra un sottoproblema, controlla la tabella.
- Se il valore non è -1 , si riutilizza il risultato salvato in $\mathcal{O}(1)$. Altrimenti, si calcola il risultato e lo si memorizza prima di ritornarlo.

Dizionario vs Tabella: L'inizializzazione di una matrice $n \times C$ costa $\mathcal{O}(nC)$, azzerando in partenza il vantaggio di non esplorare tutta la tabella. L'implementazione in linguaggi come Python avviene spesso utilizzando un **dizionario** (Hash Table). In questo modo, l'inizializzazione è istantanea e il costo computazionale scende a $\mathcal{O}(\min(2^n, nC))$. In Python, questo processo può essere del tutto automatizzato tramite un decoratore (es. `@memo` o `@lru_cache`) posto sopra la funzione ricorsiva originaria.

44.2 Variante dello Zaino, Senza Limiti (Unbounded Knapsack)

Definizione

Dato uno zaino di capacità C e n oggetti caratterizzati da peso w e profitto p , individuare il massimo profitto, **senza porre limiti al numero di volte che un oggetto può essere selezionato**.

La formula ricorsiva cambia lievemente: se decido di prendere l'oggetto i -esimo, continuo a valutarlo riducendo la capacità ma senza esaurire l'oggetto (l'indice rimane i , non $i-1$):

$$DP[i][c] = \max(DP[i][c - w[i-1]] + p[i-1], DP[i-1][c])$$

44.2.1 Semplificazione ad un vettore unidimensionale

In questo specifico caso, poiché la disponibilità degli oggetti è illimitata, si può eliminare la dimensione degli oggetti i dallo stato e definire la tabella in base alla sola capacità. Definiamo $DP[c]$ come il massimo profitto ottenibile da uno zaino di capacità $c \leq C$:

$$DP[c] = \begin{cases} 0 & c = 0 \\ \max_{w[i-1] \leq c} \{DP[c - w[i-1]] + p[i-1]\} & c > 0 \end{cases}$$

```

% Pseudocodice per Zaino Senza Limiti con Memoization
int knapsackRec(int[] w, int[] p, int n, int c, int[] DP, int[] pos)
if c == 0 then return 0
if DP[c] < 0 then
DP[c] = 0
for i = 1 to n do
if w[i-1] <= c then
int val = knapsackRec(w, p, n, c-w[i-1], DP, pos) + p[i-1]
if val >= DP[c] then
DP[c] = val
pos[c] = i - 1 % Salvo l'indice per ricostruire la soluzione
return DP[c]

```

Vantaggi e Svantaggi: La complessità in spazio cala drasticamente a $\Theta(C)$. Tuttavia, il compattamento dell'informazione in un solo vettore unidimensionale rende necessario memorizzare un vettore ausiliario pos (che tiene traccia dell'ultimo oggetto aggiunto) per poter ricostruire la soluzione ottima.

45 La Sottosequenza Comune Massimale (LCS)

Definizioni

- **Sottosequenza:** P è sottosequenza di T se è ottenuta da T rimuovendo zero o più elementi, mantenendo il medesimo ordine per i rimanenti (non necessariamente contigui). Nota: la sequenza vuota è sottosequenza di ogni sequenza.
- **Sottosequenza Comune Massimale (LCS):** Date due sequenze T e U , una sequenza X è la LCS se è sottosequenza di entrambe, e non esiste un'altra sottosequenza comune Y tale che $|Y| > |X|$.

Il problema di trovare la LCS ha numerose applicazioni in bioinformatica (valutare similarità tra sequenze di DNA) e nel campo informatico (distanza di edit). Una soluzione di forza bruta prevede di testare tutte le 2^n sottosequenze possibili di T e verificarne l'appartenenza a U , con complessità asintotica inaccettabile $\Theta(2^n(m+n))$.

45.1 Sottostruttura Ottima e Formula Ricorsiva

Il problema presenta una sottostruttura ottima valutando i prefissi delle sequenze. Sia $T(i)$ il prefisso composto dai primi i caratteri di T , e $U(j)$ il prefisso dei primi j caratteri di U . Esistono due macro-casi per calcolare la LCS:

1. **I caratteri finali coincidono** ($t_i = u_j$). La LCS si ottiene prendendo la LCS dei prefissi di dimensione $i-1$ e $j-1$, concatenando ad essa il carattere t_i .

$$LCS(T(i), U(j)) = LCS(T(i-1), U(j-1)) \oplus t_i$$

2. **I caratteri finali non coincidono** ($t_i \neq u_j$). Bisogna calcolare le LCS scartando una volta il carattere di T e una volta quello di U , e prendere la più lunga tra le due.

$$LCS(T(i), U(j)) = \text{longest}(LCS(T(i-1), U(j)), LCS(T(i), U(j-1)))$$

45.1.1 Algoritmo Dinamico

Da queste definizioni deriva l'algoritmo di calcolo *Bottom-Up* per trovare solo la **lunghezza** della LCS. Inizializziamo a 0 la prima riga e la prima colonna (i casi in cui uno dei prefissi è vuoto).

```
int lcs(item[] T, item[] U, int n, int m)
int[][] DP = new int[0..n][0..m]
for i = 0 to n do DP[i][0] = 0
for j = 0 to m do DP[0][j] = 0
```

```
for i = 1 to n do
```

```

for j = 1 to m do
    if T[i-1] == U[j-1] then
        DP[i][j] = DP[i-1][j-1] + 1
    else
        DP[i][j] = max(DP[i-1][j], DP[i][j-1])
return DP[n][m]

```

Complessità computazionale: $\mathcal{O}(mn)$.

45.2 Ricostruzione della Soluzione e Ottimizzazione

- **Ricostruzione:** Partendo dalla cella $DP[n][m]$, si risale l'albero delle decisioni. Se i caratteri corrispondenti in T e U sono uguali (il valore deriva dalla diagonale in alto a sinistra $i-1, j-1$), quel carattere fa parte della LCS. Se sono diversi, ci si sposta verso la cella (sopra o a sinistra) che ha originato il valore massimo. Questa operazione ha costo $\mathcal{O}(n+m)$.
- **Risparmio Spaziale:** Se è richiesta solo la misura della similitudine (la lunghezza della LCS) e non l'estrazione testuale della stringa, è possibile ottimizzare pesantemente lo spazio. Siccome il calcolo della riga i richiede unicamente l'accesso alla riga precedente $i-1$, è sufficiente conservare in memoria solo 2 righe (DP' e DP), abbassando il costo spaziale da $\mathcal{O}(mn)$ a $\mathcal{O}(\min(n, m))$.

45.3 Reality Check: Il tool diff

Un'applicazione pratica onnipresente dell'algoritmo LCS è la utility Unix `diff`. Esamina file di testo ricercando le differenze a livello di righe (considerate come simboli atomici per il calcolo). Poiché confrontare intere righe stringa per stringa sarebbe oneroso, `diff` utilizza internamente funzioni hash per convertire ogni riga in un numero da confrontare. Adotta inoltre euristiche preparatorie, scartando d'ufficio i prefissi e i suffissi identici dei file per concentrare il pesante calcolo LCS (il motore centrale) solo sui blocchi testuali centrali variati.

46 Programmazione Dinamica: Applicazioni Avanzate

46.1 Prodotto di Catena di Matrici

Definizione del Problema

Data una sequenza di n matrici $A_0, A_1, \dots, A_{n-1}$, compatibili due a due al prodotto, vogliamo calcolare il loro prodotto totale. Poiché il prodotto di matrici gode della proprietà associativa (es. $(A \cdot B) \cdot C = A \cdot (B \cdot C)$), l'ordine in cui si eseguono le moltiplicazioni (la *parentesizzazione*) altera drasticamente il numero di operazioni necessarie. **Obiettivo:** Calcolare il prodotto delle n matrici impiegando il più basso numero possibile di moltiplicazioni scalari.

46.1.1 Parentesizzazione e Numeri di Catalan

Il numero di parentesizzazioni possibili $NP[n]$ cresce in modo esponenziale, seguendo i Numeri di Catalan:

$$NP[n] = C(n-1) = \frac{1}{n} \binom{2n-2}{n-1} = \Theta\left(\frac{4^n}{n\sqrt{n}}\right)$$

Poiché $NP[n] = \Omega(2^n)$, un approccio di forza bruta che provi tutte le combinazioni è computazionalmente inaccettabile.

46.1.2 Sottostruttura Ottima e Formula Ricorsiva

Sia $A[i...j]$ una sottosequenza del prodotto di matrici e $P[i...j]$ una sua parentesizzazione ottima. Deve esistere un "ultimo prodotto" a un certo indice k tale che $P[i...j] = P[i...k] \cdot P[k+1...j]$. **Teorema:** Se la parentesizzazione globale è ottima, anche $P[i...k]$ e $P[k+1...j]$ devono obbligatoriamente essere parentesizzazioni ottime per i rispettivi sottoproblemi.

Sia $DP[i][j]$ il minimo numero di moltiplicazioni scalari necessarie per calcolare $A[i...j]$ e $c[i]$ il vettore delle dimensioni delle matrici (A_i ha $c[i]$ righe e $c[i+1]$ colonne). La formula ricorsiva è:

$$DP[i][j] = \begin{cases} 0 & \text{se } i = j \\ \min_{i \leq k < j} \{DP[i][k] + DP[k+1][j] + c[i] \cdot c[k+1] \cdot c[j+1]\} & \text{se } i < j \end{cases}$$

46.1.3 Algoritmo Bottom-Up

Per risolvere il problema si utilizzano due matrici: DP per i costi e $last$ per registrare l'indice k che ha generato il costo minimo (necessario per ricostruire la soluzione). **Attenzione:** Non si può riempire la tabella riga per riga, ma bisogna procedere per diagonali (indicizzate da h), risolvendo prima le catene di lunghezza 2, poi 3, e così via.

```

% Programmazione dinamica per la catena di matrici
computePar(int[] c, int n)
int[] [] DP = new int[0...n-1][0...n-1]
int[] [] last = new int[0...n-1][0...n-1]

% Inizializza la diagonale principale (catene di lunghezza 1)
for i = 0 to n - 1 do DP[i][i] = 0

% h è la lunghezza della catena meno 1
for h = 1 to n - 1 do
    for i = 0 to n - h - 1 do
        int j = i + h
        DP[i][j] = +
        % Prova tutti i possibili punti di split k
        for k = i to j - 1 do
            int temp = DP[i][k] + DP[k+1][j] + c[i]*c[k+1]*c[j+1]
            if temp < DP[i][j] then
                DP[i][j] = temp
                last[i][j] = k
return DP[0][n-1]

```

Complessità computazionale: $\mathcal{O}(n^3)$, in quanto ogni cella della sottomatrice superiore (che sono $\Theta(n^2)$) richiede un tempo $\mathcal{O}(n)$ per ispezionare tutti i possibili indici k .

46.2 Insieme Indipendente di Intervalli Pesati

Definizione del Problema

Dati n intervalli temporali $[s[0], f[0]), \dots, [s[n-1], f[n-1])$ della retta reale, ognuno con un profitto associato $w[i]$. Due intervalli sono disgiunti se $f[j] \leq s[i]$ oppure $f[i] \leq s[j]$. **Obiettivo:** Trovare un sottoinsieme di intervalli **disgiunti** tra loro che massimizzi la somma dei profitti.

46.2.1 Pre-elaborazione Necessaria

Per applicare correttamente la programmazione dinamica in questo scenario, è obbligatorio effettuare una **pre-elaborazione** dell'input:

1. Ordinare gli intervalli per tempi di fine non decrescenti ($f[0] \leq f[1] \leq \dots \leq f[n-1]$). Costo $\mathcal{O}(n \log n)$.
2. Pre-calcolare un vettore dei predecessori $\text{pred}[i] = j$, dove $j < i$ è il *massimo* indice tale per cui l'intervallo $j-1$ è compatibile con l'intervallo $i-1$ ($f[j-1] \leq s[i-1]$). Questo può essere calcolato in $\mathcal{O}(n \log n)$ usando la ricerca binaria.

46.2.2 Formula DP e Algoritmo

$DP[i]$ contiene il profitto massimo ottenibile considerando i primi i intervalli ordinati.

$$DP[i] = \begin{cases} 0 & \text{se } i = 0 \\ \max(DP[i-1], DP[\text{pred}[i]] + w[i-1]) & \text{se } i > 0 \end{cases}$$

L'algoritmo valuta se "non prendere" l'intervallo (ereditando $DP[i-1]$) oppure "prenderlo" (sommando il suo profitto al miglior profitto ottenibile prima del suo inizio, $DP[\text{pred}[i]]$).

Complessità Totale: Ordinamento $\mathcal{O}(n \log n)$ + calcolo pred $\mathcal{O}(n \log n)$ + riempimento tabella e ricostruzione $\mathcal{O}(n) \implies \mathcal{O}(n \log n)$.

46.3 String Matching Approssimato

Definizione del Problema

Date una stringa pattern P di lunghezza m e un testo T di lunghezza n . Un'occorrenza k -approssimata di P in T è una copia che ammette k "errori" di tre tipi:

1. **Sostituzione:** I caratteri corrispondenti in P e T sono diversi.
2. **Inserimento:** Un carattere di P manca in T .
3. **Cancellazione:** Un carattere di T è di troppo rispetto a P .

Obiettivo: Trovare un'occorrenza k -approssimata di P in T con k minimo. È alla base del calcolo della distanza di edit (Levenshtein) e di tool come diff.

46.3.1 Sottostruttura Ottima

Sia $DP[i][j]$ il minimo numero di errori k per far combaciare il prefisso $P(i)$ con un suffisso del prefisso $T(j)$ terminante in $T[j-1]$. Si definisce un indicatore δ che vale 0 se i caratteri $P[i-1]$ e $T[j-1]$ coincidono, e 1 se sono diversi (errore di sostituzione).

$$DP[i][j] = \begin{cases} 0 & \text{se } i = 0 \\ i & \text{se } j = 0 \\ \min \begin{cases} DP[i-1][j-1] + \delta \\ DP[i-1][j] + 1 \\ DP[i][j-1] + 1 \end{cases} & \text{altrimenti} \end{cases}$$

46.3.2 Algoritmo e Take-home Message

```
% Algoritmo per String Matching Approssimato
int stringMatching(item[] P, item[] T, int m, int n)
int[] [] DP = new int[0...m][0...n]
for j = 0 to n do DP[0][j] = 0
for i = 1 to m do DP[i][0] = i

for i = 1 to m do
  for j = 1 to n do
    int delta = iif(P[i-1] == T[j-1], 0, 1)
    DP[i][j] = min( DP[i-1][j-1] + delta,
                   DP[i-1][j] + 1,
                   DP[i][j-1] + 1 )

% Ricerca della soluzione nell'ultima RIGA
int pos = 1
for j = 2 to n do
```

```
    if DP[m][j] < DP[m][pos] then pos = j
return pos - 1
```

Take-home message: Non è garantito che la soluzione ottima si trovi sempre nella casella in basso a destra ($DP[m][n]$). In questo caso specifico, il pattern potrebbe fare match in *qualsiasi* punto del testo T , pertanto la soluzione va cercata scansionando tutta l'ultima riga ($DP[m][j]$) alla ricerca del valore minimo.